DEVELOPMENT OF A FAULT-TOLERANT ROCKET AVIONICS HARDWARE SYSTEM FOR FLIGHT CONTROL AND AEROSOL MONITORING

PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Electronics and Communication Engineering

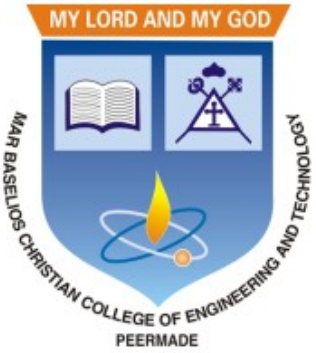
by

AADIL MUHAMMED (MBC22EC001)

BEVAL PHILIP MATHEW (MBC22EC018)

RAHUL B (MBC22EC045)

SIBI B JOHN (MBC22EC054)



MAR BASELIOS CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY, PEERMADE

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

APRIL 2026

DEVELOPMENT OF A FAULT-TOLERANT ROCKET AVIONICS HARDWARE SYSTEM FOR FLIGHT CONTROL AND AEROSOL MONITORING

PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Electronics and Communication Engineering

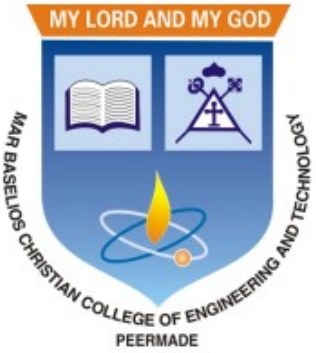
by

AADIL MUHAMMED (MBC22EC001)

BEVAL PHILIP MATHEW (MBC22EC018)

RAHUL B (MBC22EC045)

SIBI B JOHN (MBC22EC054)



MAR BASELIOS CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY, PEERMADE

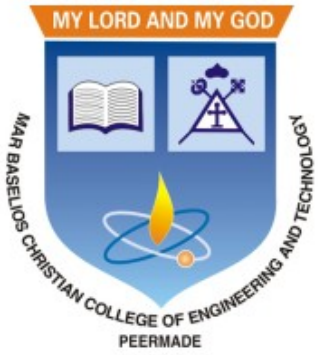
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

APRIL 2026

MAR BASELIOS CHRISTIAN COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

2025-2026



CERTIFICATE

This is to certify that this report entitled **"Development of a Fault-Tolerant Rocket Avionics Hardware System for Flight Control and Aerosol Monitoring"** submitted by **AADIL MUHAMMED (MBC22EC001), BEVAL PHILIP MATHEW (MBC22EC018), RAHUL B (MBC22EC045),** and **SIBI B JOHN (MBC22EC054)** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering, is a bonafide report of the Project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof.Nimisha George	**Prof.Elias Janson K**	**Dr.Oommen Tharakan K.T**
Project Guide	Project Coordinator	Head of the Department
Assistant Professor	Associate Professor	Professor
Department of ECE	Department of ECE	Department of ECE

ABSTRACT

The hardware design and integration of Basilian-01 present a model rocket avionics system developed for reliable flight monitoring, effective recovery control, and atmospheric aerosol data acquisition. The system is centered on a dual- redundant embedded hardware architecture to enhance mission reliability and eliminate single-point failures during critical flight phases.Two independent ESP32-based flight control units are employed, each interfaced with dedicated barometric sensors for altitude estimation, ensuring fault tolerance through redundant sensing and control paths. The avionics hardware integrates multiple subsystems including barometric altitude sensors, an inertial measurement unit (IMU), GPS module, LoRa-based telemetry transceiver, SD card data logger, servo-based parachute deployment mechanism, and an aerosol sensing module for in-flight atmospheric particulate measurement. Aerosol data is collected during ascent and descent phases to enable analysis of particulate concentration variations with altitude. All recovery-critical operations, such as apogee detection and parachute deployment, are designed to operate autonomously on-board, independent of ground systems. Emphasis is placed on robust power distribution, signal interfacing, sensor integration, and redundancy-oriented hardware design to ensure safe operation under dynamic flight conditions. The hardware architecture demonstrates strong suitability for experimental aerospace platforms that require reliable flight control along with atmospheric and aerosol data acquisition, making it well-suited for educational, research, and environmental monitoring applications using model rockets.

DECLARATION

We hereby declare that the project report "**DEVELOPMENT OF A FAULT-TOLERANT ROCKET AVIONICS HARDWARE SYSTEM FOR FLIGHT CONTROL AND AEROSOL MONITORING**", submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Dr. Oommen Tharakan K.T** and **Prof. Nimisha George**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

PEERMADE,

10-04-2026

AADIL MUHAMMED

BEVAL PHILIP MATHEW

RAHUL B

SIBI B JOHN

ACKNOWLEDGEMENT

We take this opportunity to express our deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to **Dr. V I George, Principal**, Mar Baselios Christian College of Engineering and Technology, Peermade, for his valuable support. We would also like to thank **Dr. Oommen Tharakan K.T., Professor and HoD** of Electronics and Communication Engineering, for providing us with all the necessary facilities and support. We would like to express our sincere gratitude to our project coordinator **Prof. Elias Janson K, Associate Professor and Vice Principal** for his support and coordination. We would like to express our sincere gratitude to our project guide **Prof. Nimisha George, Assistant Professor**, for the guidance throughout the project. Finally, We thank our families and friends who contributed to the successful fulfillment of this project work.

AADIL MUHAMMED
BEVAL PHILIP MATHEW
RAHUL B
SIBI B JOHN

CONTENTS

LIST OF TABLES

ABBREVIATIONS

IMU	– Inertial Measurement Unit
DOF	– Degrees of Freedom
MCU	– Microcontroller Unit
MEMS	– Micro-Electro-Mechanical Systems
PWM	– Pulse Width Modulation
GPS	– Global Positioning System
RF	– Radio Frequency
LoRa	– Long Range (Wireless Communication Technology)
UBEC	– Universal Battery Elimination Circuit
Li-Po	– Lithium Polymer Battery
SD Card	– Secure Digital Card
PCB	– Printed Circuit Board
ADC	– Analog-to-Digital Converter
I^2C	– Inter-Integrated Circuit
SPI	– Serial Peripheral Interface
UART	– Universal Asynchronous Receiver Transmitter
PID	– Proportional–Integral–Derivative

CHAPTER 1
INTRODUCTION

Sounding rockets and model rockets have become increasingly important platforms for low-cost aerospace experimentation, atmospheric studies, and avionics system validation. Unlike large-scale launch vehicles, model rockets provide an accessible testbed for evaluating embedded hardware performance, sensor integration, telemetry systems, and recovery mechanisms under dynamic flight conditions. However, reliable operation of on-board avionics remains a major challenge due to high acceleration, vibration, rapid altitude variation, and strict size, weight, and power constraints.

Modern rocket avionics systems are expected to perform multiple critical functions such as altitude estimation, flight state detection, telemetry transmission, data logging, and recovery system actuation. Failure in any of these subsystems can result in mission loss, unsafe recovery, or loss of experimental data. Conventional single-controller avionics architectures suffer from single-point failures, making redundancy and fault tolerance essential design requirements, even for educational and experimental rocket platforms.

In addition to flight control and recovery, rockets offer a unique opportunity for short-duration atmospheric measurements. Vertical profiling using rockets enables the collection of environmental and aerosol data across different altitude layers within a single flight. Aerosols play a significant role in atmospheric chemistry, air quality, and climate behavior, yet most aerosol monitoring systems are ground-based or rely on expensive aerial platforms. Integrating aerosol sensing hardware into a model rocket provides a low-cost and reusable method for studying particulate variation with altitude.

The Basilian-01 project focuses on the hardware design and integration of a dual-redundant rocket avionics system with aerosol sensing capability. The system employs redundant embedded controllers, multiple sensors for altitude and motion estimation, telemetry modules for real-time data transmission, onboard data logging, and a servo-based parachute deployment mechanism for safe recovery. Emphasis is placed on robust hardware architecture, power management, sensor interfacing, and reliability under flight conditions.

By combining avionics hardware design with atmospheric aerosol measurement, the project aims to demonstrate a multifunctional rocket platform suitable for educational aerospace missions and experimental environmental data collection. The proposed system highlights the feasibility of low-cost, reliable rocket avionics with added scientific payload capability, contributing toward accessible aerospace experimentation and environmental monitoring applications.

CHAPTER 2
LITERATURE SURVEY

The paper titled "Development of a Servo-Based Model Rocket and Flight Behavior Analysis Using IMU Data" presents a comprehensive study on the design, development, and experimental validation of a servo-controlled model rocket with an emphasis on flight behavior analysis using inertial measurement unit (IMU) data[**1**]. The main objective of the work is to demonstrate how embedded avionics hardware can be employed to monitor, record, and analyze the dynamic motion of a rocket during various phases of flight. The authors focus on understanding the complex interaction between aerodynamic forces, mechanical actuation, and inertial response under real flight conditions.The avionics system integrates a microcontroller with a multi-axis IMU capable of measuring linear acceleration, angular velocity, and orientation. These parameters are continuously acquired throughout the flight, allowing for detailed post-flight analysis of rocket behavior. The study explains how IMU data can be used to identify critical flight events such as launch detection, peak acceleration, apogee, and descent initiation. A servo motor is incorporated into the system to actuate control surfaces or recovery-related mechanisms. The interfacing of the servo with the controller is discussed in detail, including timing constraints, power requirements, and mechanical robustness.Flight tests are conducted to validate the proposed design, and the results indicate that the servo mechanism performs reliably despite exposure to vibration, shock, and rapid changes in motion. The IMU data obtained from the experiments is analyzed to evaluate flight stability, rotational motion, and attitude variation. The authors also discuss inherent limitations of IMU-based systems, such as sensor noise, drift, and calibration errors, and emphasize the need for signal filtering and data processing techniques.Although the system successfully demonstrates servo actuation and IMU-based flight analysis, it relies on a single flight controller and lacks redundancy, telemetry, and environmental sensing. Despite these limitations, the paper provides a strong technical foundation for advanced rocket avionics development. The concepts presented directly influence the Basilian-01 project, which extends this work by incorporating redundant avionics hardware, telemetry, autonomous recovery mechanisms, and aerosol sensing capabilities.

The paper titled "Design and Implementation of a Flight Computer for Sounding Rockets (S3FC)" presents a comprehensive study on the design methodology, subsystem integration, and operational challenges associated with space-capable sounding rockets developed by collegiate teams [**2**]. The main objective of the work is to highlight the importance of robust avionics architecture in ensuring reliable rocket operation under extreme flight conditions. The authors emphasize that sounding rockets, despite their rela-

tively small size, are subjected to high acceleration forces, intense vibration, and rapidly changing atmospheric environments, making avionics system reliability a critical factor in determining mission success or failure. The avionics system is described in terms of its major subsystems, including the flight computer, sensor modules, telemetry systems, and recovery mechanisms, which operate in a tightly coupled manner where each component contributes to overall mission functionality. The study explains how interdependency among subsystems introduces vulnerability, as failures in power distribution or communication links can propagate across the system and lead to complete mission failure, supported by real-world examples from collegiate rocket missions illustrating issues such as power brownouts, connector loosening due to vibration, sensor malfunctions, and telemetry dropouts. A modular avionics design approach is strongly emphasized, where separating critical subsystems and enabling independent testing improves reliability, facilitates fault isolation, and simplifies troubleshooting, while onboard data logging is highlighted as a critical backup mechanism when real-time telemetry becomes unreliable during high-altitude or high-speed flight. The findings reinforce the necessity of adopting professional aerospace design practices even in student-level projects, including redundancy, fail-safe recovery logic, and extensive ground testing to minimize system-level risks. Although the study provides a strong foundation in avionics system design and reliability engineering, it does not extensively address advanced features such as autonomous decision-making, environmental sensing, or multi-layer redundancy architectures. The concepts presented in this work directly influence the Basilian-01 project, which extends these principles by incorporating modular hardware architecture, enhanced data logging strategies, redundant avionics systems, and reliability-focused design improvements aimed at improving mission robustness and operational safety.

The paper titled "A Sounding Rocket as a Test Bench for Cost-Effective Aerospace Measurements" focuses on the development of fault-tolerant embedded systems aimed at enhancing the safety and reliability of rocket flight control and parachute deployment mechanisms [**3**]. The main objective of the work is to address the limitations of conventional rocket avionics systems, particularly the issue of single-point failure, where the malfunction of a single controller or sensor can result in complete mission failure and unsafe recovery. To overcome this limitation, the authors propose a redundant embedded controller architecture in which multiple independent microcontrollers continuously monitor flight parameters and execute recovery logic. Each controller independently evaluates critical conditions such as altitude, acceleration, and elapsed flight time, ensuring reliable parachute deployment even if one controller becomes inoperative. The study provides a detailed analysis of potential failure scenarios, including processor lock-up, sensor disconnection, power interruption, and communication failure, and evaluates system behavior under such conditions. Simulation results and experimental validations

demonstrate that fault-tolerant architectures significantly improve recovery success rates compared to traditional single-controller designs. The authors also discuss practical challenges associated with redundancy, including synchronization between controllers and the possibility of conflicting deployment commands, and propose solutions such as simplified logic structures and priority-based decision-making schemes to mitigate these issues. The findings establish redundancy as a fundamental design principle rather than an optional enhancement in aerospace systems. The concepts presented in this work directly influence the Basilian-01 project, which adopts a dual-redundant avionics hardware architecture for safety-critical operations, particularly in functions such as apogee detection and parachute deployment.

The paper titled "Low-Cost Avionics Systems for Small-Scale Rocket Applications" investigates the feasibility of using rocket platforms as tools for atmospheric and aerosol data acquisition [**4**]. The main objective of the study is to explore the capability of rocket-based systems to perform vertical profiling of aerosol concentrations, which are otherwise difficult to measure using conventional ground-based monitoring techniques. The authors highlight that aerosols play a significant role in atmospheric chemistry, air quality, and climate dynamics, and emphasize the need for efficient methods to capture their distribution across different altitude layers within a short time duration. The study presents the integration of compact aerosol sensors into a rocket payload and examines key engineering challenges such as vibration sensitivity, airflow disturbances, sensor response latency, and power stability. Particular attention is given to sensor placement and protective shielding to minimize measurement distortion caused by high-speed airflow and mechanical vibrations during flight. Experimental flight results demonstrate that reliable aerosol data can be obtained when these design considerations are properly addressed. The authors further emphasize the importance of synchronizing aerosol measurements with altitude, time, and flight phase data, as accurate correlation between environmental sensing and avionics data is essential for meaningful analysis. The findings validate the feasibility of low-cost rocket-based aerosol monitoring systems and highlight their potential applications in atmospheric research. The concepts presented in this work directly influence the Basilian-01 project, which integrates aerosol sensing hardware into a model rocket avionics system to extend its functionality beyond flight testing into environmental data collection.

The paper titled "Design of Redundant Avionics Systems for Experimental Rockets" presents the design and implementation of an embedded hardware platform for aerospace environmental sensing and telemetry applications [**5**]. The main objective of the work is to develop a compact, lightweight, and power-efficient system capable of operating reliably under harsh aerospace conditions such as high vibration, temperature variation, pressure

changes, and electromagnetic interference. The proposed system integrates multiple environmental sensors, including temperature, pressure, and gas or particulate sensors, with an embedded microcontroller for data acquisition and processing. A wireless telemetry module is used for real-time data transmission to a ground station, while onboard data logging is implemented as a backup to prevent data loss during communication failures. The study emphasizes the importance of proper sensor interfacing, signal conditioning, and analog-to-digital conversion to reduce noise and improve measurement accuracy. Significant attention is given to power management and signal integrity, where techniques such as regulated power supply, decoupling, grounding, and electromagnetic shielding are shown to enhance system stability. Telemetry performance is evaluated through testing, demonstrating that low-power long-range communication modules can achieve reliable data transmission with minimal packet loss when properly configured. The study also highlights the importance of antenna placement, transmission optimization, and data rate selection in maintaining communication reliability. The results establish that combining real-time telemetry with onboard data logging improves both mission monitoring and post-flight analysis. The concepts presented in this work directly influence the Basilian-01 project, particularly in terms of sensor integration, telemetry architecture, and power system reliability.

The paper titled "Node Replacement Method for Disaster-Resilient Wireless Sensor Networks" focuses on the development of a high-reliability telemetry and data acquisition system intended for suborbital vehicles such as sounding rockets and experimental launch platforms [**6**]. The main objective of the study is to ensure reliable data transmission and preservation under challenging flight conditions where communication opportunities are limited. The authors highlight key challenges such as communication dropouts, signal attenuation, and data corruption during high-speed and high-altitude operations. To address these issues, the proposed system adopts a hybrid data handling approach that combines real-time telemetry with onboard data logging. Real-time telemetry enables continuous monitoring of flight parameters such as altitude, velocity, and system health, while onboard storage ensures complete data retention even if communication links fail. The study describes the telemetry architecture in detail, including packet structure, error detection mechanisms, and synchronization techniques for reliable data transfer. Experimental evaluations demonstrate that low-power long-range communication modules can achieve stable transmission with minimal packet loss when properly configured. The authors also emphasize the importance of antenna placement, transmission power optimization, and data rate selection in improving communication reliability. The results establish that redundancy in both data acquisition and transmission significantly enhances mission success and post-flight analysis capability. The concepts presented in this work directly support the telemetry and data acquisition strategy implemented in the

Basilian-01 avionics system.

The paper titled "Implementation of LoRa-Based Telemetry Systems for Long-Range Aerospace Applications" presents a detailed study on the design and implementation of an integrated multi-sensor avionics architecture aimed at improving the accuracy and reliability of flight monitoring in sounding rocket missions [**7**]. The main objective of the work is to overcome the limitations of traditional single-sensor-based systems, which are often affected by noise, drift, sensor saturation, and environmental disturbances, leading to inaccurate flight state estimation. To address these challenges, the authors propose a multi-sensor approach that combines data from inertial measurement units (IMUs), barometric pressure sensors, and global positioning system (GPS) receivers, where each sensor provides complementary information for accurate flight analysis. IMUs offer high-rate motion data, barometric sensors provide altitude estimation, and GPS modules supply absolute position and velocity information, enabling reliable detection of critical flight events such as launch, apogee, and descent. The study emphasizes the importance of sensor synchronization and data fusion techniques, where proper time-stamping and alignment of sensor data reduce inconsistencies and improve overall system accuracy. Experimental results demonstrate that the integrated sensor network significantly enhances trajectory reconstruction and flight phase identification compared to single-sensor systems. The authors also discuss practical challenges such as increased computational load, power consumption, and hardware complexity, but conclude that the improvement in reliability and accuracy outweighs these limitations. The concepts presented in this work directly support the Basilian-01 project, which adopts a multi-sensor avionics approach to improve robustness, reduce dependency on individual sensors, and enhance overall mission reliability.

The paper titled "Embedded Sensor Fusion Techniques for Flight State Estimation in Small Rockets" investigates the critical role of power distribution and signal integrity in ensuring reliable operation of aerospace embedded systems [**8**]. The main objective of the study is to address power-related challenges that arise in aerospace environments, where high vibration, temperature variation, electromagnetic interference, and transient disturbances can significantly affect system performance. The authors identify power instability as a major cause of avionics failure, leading to issues such as processor resets, sensor malfunction, communication errors, and unintended actuator behavior. The study provides a detailed analysis of common power system problems, including voltage fluctuations, transient spikes, ground loops, and electromagnetic interference, and explains their impact on both analog and digital subsystems. To mitigate these challenges, the paper proposes several design strategies, including the use of regulated power supplies, proper grounding techniques, decoupling and bulk capacitors, transient voltage suppres-

sion devices, and electromagnetic shielding. The importance of PCB layout design is also emphasized, with recommendations such as separating analog and digital grounds, minimizing loop areas, and optimizing current paths to reduce noise coupling. Experimental results demonstrate that systems designed with proper power management and signal integrity techniques exhibit improved stability, reduced system resets, enhanced sensor accuracy, and reliable communication performance under dynamic conditions. The authors conclude that power system design must be treated as a fundamental aspect of avionics development. The concepts presented in this work directly influence the Basilian-01 project, which incorporates robust power regulation, grounding strategies, and noise suppression techniques to ensure stable operation of flight control, telemetry, and environmental sensing systems throughout the mission.

The paper titled "Design and Validation of Redundant Recovery Systems for Model Rockets" presents a detailed study on the development of an embedded flight computer for sounding rocket applications [**9**]. The main objective of the work is to demonstrate a reliable and low-cost avionics solution capable of performing critical flight control and data handling tasks under harsh operational conditions. The authors highlight that commercial off-the-shelf microcontrollers, when combined with robust design practices, can meet the reliability requirements of sounding rocket missions. The proposed system integrates multiple sensors, including barometric pressure sensors, inertial measurement units, and a GPS module, to enable accurate flight state estimation and autonomous detection of key flight events such as launch, burnout, apogee, and descent. Based on these detections, the system executes recovery logic for parachute deployment, supported by filtering techniques and apogee detection algorithms to reduce false triggering caused by sensor noise and transient disturbances. The study emphasizes data integrity through the use of cyclic redundancy check (CRC) mechanisms for reliable telemetry transmission and onboard non-volatile memory for secure data logging in case of communication failure. Hardware design aspects such as PCB layout, power regulation, and component selection are also discussed to enhance system robustness. Experimental evaluations demonstrate that the system reliably performs flight event detection, telemetry communication, and recovery actuation under simulated conditions. The authors conclude that a well-designed embedded flight computer can significantly improve mission reliability without increasing system complexity. The concepts presented in this work directly support the Basilian-01 project, particularly in terms of autonomous flight logic, sensor filtering, data integrity, and recovery system implementation.

The paper titled "Power Management and Reliability Enhancement in Embedded Aerospace Systems" explores the use of sounding rockets as flexible and cost-effective platforms for validating aerospace technologies and atmospheric measurement systems

[**10**]. The main objective of the study is to demonstrate that sounding rockets can serve as modular and reusable test beds for experimental hardware under real flight conditions, offering a low-cost alternative to traditional aerospace testing platforms. The authors describe the development of a sounding rocket demonstrator designed to support modular payload integration, allowing various avionics systems, sensors, and experimental components to be tested without requiring complete system redesign. The study emphasizes the importance of modularity in reducing development time and enabling rapid iteration of experimental payloads. A key focus is placed on the integration of environmental measurement systems, where sensors are used to capture parameters such as pressure, temperature, acceleration, and other atmospheric variables. The authors discuss challenges including sensor calibration, data synchronization, and environmental disturbances during flight, and highlight that reliable data acquisition can be achieved through proper hardware integration and data handling techniques. The paper also addresses recovery system design and data preservation, ensuring safe retrieval of the payload for reuse and post-flight analysis. Experimental results validate that sounding rockets are effective platforms for short-duration atmospheric measurements and aerospace system testing at reduced cost. The concepts presented in this work directly support the Basilian-01 project, particularly in terms of modular payload design, cost-effective experimentation, and the integration of environmental sensing capabilities such as aerosol monitoring.

CHAPTER 3
SYSTEM SPECIFICATIONS AND DESIGN JUSTIFICATION

3.1 Overview

This chapter presents the technical specifications and design rationale of the major subsystems incorporated in the proposed Basilian-01 rocket avionics architecture. The selection of each hardware component has been carried out after careful consideration of performance requirements, operational constraints, environmental conditions, power efficiency, reliability, weight limitations, and overall system compatibility. Since the rocket operates in a dynamic and vibration-intensive environment, the design emphasizes stability, fault tolerance, low latency response, and energy optimization. The avionics system is divided into onboard and ground control segments, ensuring modularity and simplified troubleshooting. The components selected are commercially available, cost-effective, and suitable for experimental academic rocketry missions. Each subsystem is justified below based on its functional necessity and engineering suitability.

3.2 Flight Controller – ESP32

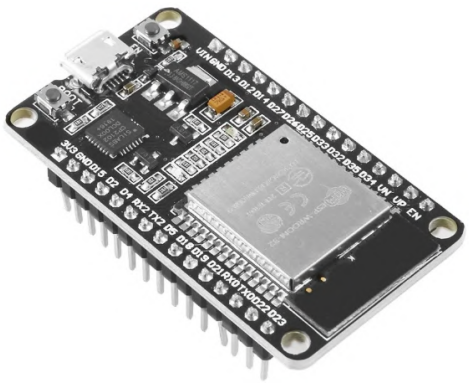


FIG 3.1:ESP-32

The ESP32 microcontroller is selected as the primary flight controller for the Basilian-01 rocket due to its high processing capability, integrated communication features, and extensive peripheral support. The ESP32 is a dual-core 32-bit microcontroller operating at clock speeds up to 240 MHz, which makes it suitable for real-time flight data acquisition and processing. In rocket applications, simultaneous handling of sensor inputs, telemetry transmission, deployment logic, and data logging is required. The dual-core architecture enables task separation, where one core can manage sensor fusion and state estimation while the other handles communication and storage operations.

Another significant advantage of the ESP32 is its wide range of supported communication protocols, including I²C, SPI, UART, ADC, and PWM. This flexibility allows seamless integration with sensors such as the MPU6050 IMU, BME280 environmental sensor, GPS module, LoRa transceiver, and SD card module. The built-in Wi-Fi and Bluetooth capabilities also provide optional wireless debugging and ground-level configuration. Furthermore, the ESP32 operates at low power levels compared to conventional high-performance microcontrollers, which is essential for battery-powered flight systems. Considering its compact size, lightweight form factor, processing power, and cost efficiency, the ESP32 provides an optimal balance between performance and practicality for experimental rocket avionics.

3.3 Communication System – LoRa (433 MHz)

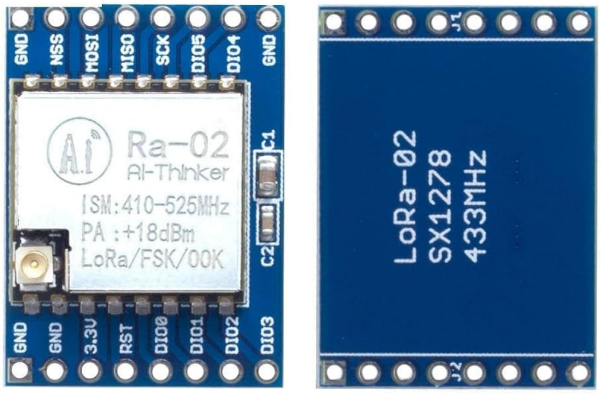


FIG 3.2:LoRa Module

Long-range telemetry communication is a critical requirement in sounding rocket missions, as the vehicle quickly exceeds the operational range of conventional wireless systems. For this reason, LoRa technology operating at 433 MHz is selected for the Basilian-01 rocket. LoRa provides long-distance communication capabilities extending several kilometers under line-of-sight conditions, making it highly suitable for open-field rocket launches. Unlike Wi-Fi or Bluetooth systems that operate at 2.4 GHz and suffer from limited range and higher power consumption, LoRa offers low data rate transmission with superior propagation characteristics and improved penetration capability.

The telemetry data transmitted during flight primarily consists of altitude, acceleration, orientation, temperature, pressure, GPS coordinates, and battery voltage. These parameters require reliability rather than high bandwidth, making LoRa an efficient choice. Operating at 433 MHz further enhances transmission range due to lower frequency propagation advantages. Additionally, LoRa employs chirp spread spectrum modulation, which improves resistance to noise and interference. Its low power consumption significantly extends battery life, which is critical for onboard avionics systems. Therefore, LoRa

provides an optimal solution for stable, energy-efficient, and long-range rocket telemetry.

3.4 Payload System – Aerosol Detection Module

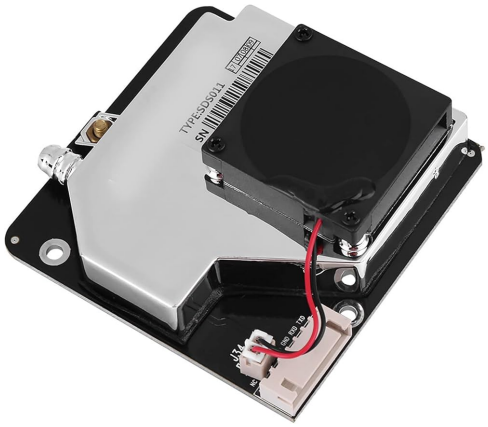


FIG 3.3:Aerosol Sensor

The payload of the Basilian-01 rocket is designed to perform aerosol concentration measurement during ascent and descent. Incorporating an aerosol sensing system enhances the scientific relevance of the mission by enabling atmospheric data collection. Aerosols play a significant role in climate behavior, air quality analysis, and environmental monitoring, and collecting vertical atmospheric data can contribute to research and experimental validation.

The aerosol detection module operates based on optical scattering principles, where suspended particulate matter interacts with a laser source, and the scattered light intensity is measured to determine particle concentration. The selected sensor is compact, lightweight, and capable of digital data output, making it compatible with the onboard microcontroller. Its low power consumption and minimal computational requirements make it suitable for rocket integration. By including an aerosol payload, the project transitions from a purely engineering demonstration to a scientifically valuable mission, thereby increasing its academic contribution.

3.5 Machine Intelligence Algorithm for Aerosol Detection

To enhance the reliability of aerosol measurements, a Machine Intelligence (MI) algorithm is proposed for onboard data processing. Raw sensor outputs may contain noise due to vibration, rapid altitude changes, and atmospheric disturbances during flight. Therefore, preprocessing techniques such as moving average filtering and noise smoothing are required to stabilize readings. The MI framework is designed to extract meaningful features such as mean particulate concentration, rate of change, and variance over time.

The algorithm may implement either adaptive thresholding or a lightweight classification model such as a decision tree or TinyML-based inference model. The objective is to categorize aerosol density into predefined levels such as low, moderate, and high concentration. This intelligent classification reduces dependency on fixed thresholds and improves detection robustness under varying environmental conditions. By incorporating machine intelligence into the payload, the system moves beyond passive data logging and introduces intelligent onboard analysis, thereby increasing innovation and research value.

3.6 Sensor Integration Module – MPU6050

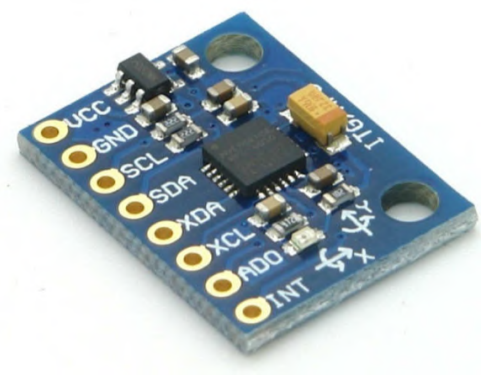


FIG 3.4:MPU6050 Sensor

The MPU6050 sensor module is integrated into the Basilian-01 payload to measure motion dynamics such as acceleration and angular velocity during flight. This sensor plays a crucial role in monitoring the rocket's orientation, stability, and trajectory throughout ascent and descent phases.

The MPU6050 combines a 3-axis accelerometer and a 3-axis gyroscope within a single compact unit, enabling precise motion tracking. It communicates with the onboard microcontroller through an I2C interface, allowing efficient data acquisition with minimal wiring. The sensor provides real-time data that can be used for flight analysis and performance evaluation.

Due to its small size, low power consumption, and high sensitivity, the MPU6050 is highly suitable for aerospace embedded applications. Incorporating this module enhances the system's capability to capture dynamic flight parameters, contributing to both engineering analysis and system validation.

3.7 Environmental Sensing Module – BME280

FIG 3.5:BME280 Sensor

The BME280 sensor module is incorporated into the Basilian-01 payload to measure atmospheric parameters such as temperature, pressure, and humidity. These measurements are essential for analyzing environmental conditions at varying altitudes during the rocket's flight.

The sensor operates using integrated sensing elements that provide high-accuracy readings with low noise. It supports both I2C and SPI communication protocols, ensuring seamless integration with the onboard microcontroller. The pressure data obtained from the sensor can also be used to estimate altitude, adding further value to the mission data.

With its compact design, low power requirements, and reliable performance, the BME280 is well-suited for embedded aerospace applications. Its inclusion enhances the payload's capability to gather comprehensive environmental data, supporting atmospheric research and system evaluation.

3.8 Data Logging Module – SD Card

FIG 3.6:SD Card Module

The SD card module is used in the Basilian-01 payload for onboard data storage, enabling reliable recording of sensor data during the rocket's flight. Since real-time data

transmission may not always be feasible, local storage ensures that all critical measurements are preserved for post-flight analysis.

The module interfaces with the microcontroller using the SPI communication protocol, allowing high-speed data transfer and efficient file handling. Sensor readings from modules such as MPU6050, BME280, and aerosol sensors are continuously logged onto the SD card in a structured format.

Its lightweight design, ease of integration, and large storage capacity make the SD card module an ideal choice for embedded systems. Incorporating this module ensures data integrity and accessibility, thereby enhancing the overall reliability and scientific value of the mission.

3.9 Power Distribution System

(a)UBEC Converter *(b)Li-Po Battery*

FIG 3.7:Power Supply Components

A stable and reliable power distribution system is essential for ensuring uninterrupted operation of avionics components. The Basilian-01 rocket utilizes regulated power modules such as UBECs to provide consistent voltage levels to microcontrollers, sensors, and servo mechanisms. During servo activation, sudden current spikes may occur, potentially causing voltage dips that could reset sensitive electronic components. To prevent such failures, separate voltage regulation paths are planned for logic circuits and actuator systems.

The power system is designed to isolate high-current components from signal-processing circuits to minimize electrical noise interference. Lithium Polymer batteries are selected due to their high energy density and lightweight characteristics, which are critical in aerospace applications. Proper voltage regulation and distribution significantly improve reliability, reduce the risk of brownout conditions, and ensure stable operation throughout

the mission duration.

3.10 Recovery and Deployment System

FIG 3.8:Servo Motor

The recovery mechanism of the Basilian-01 rocket is based on a servo-controlled deployment system. Unlike pyrotechnic deployment methods, which involve explosive charges and increased safety risks, servo-based systems offer mechanical reliability and reusability. The servo mechanism is programmed to activate based on altitude or flight-state detection logic derived from sensor data. This controlled approach ensures precise parachute deployment at apogee.

To enhance safety, the design incorporates redundancy through backup deployment logic. In case the primary trigger fails, an alternate condition-based trigger can initiate recovery. This redundancy improves mission success probability and minimizes structural damage upon landing. The servo-based recovery system is particularly suitable for academic and experimental rocket platforms where safety and reusability are priorities.

3.11 Power and Imaging Subsystem

The power and imaging subsystem plays a crucial role in ensuring continuous operation and real-time visual data acquisition during the mission. The system is powered by a Li-Po (Lithium Polymer) battery, which serves as the primary energy source due to its high energy density, lightweight nature, and ability to deliver high discharge currents. Since different components in the system require regulated voltage levels, a UBEC (Universal Battery Elimination Circuit) is employed. The UBEC efficiently steps down and stabilizes the battery voltage to a constant 5V or 3.3V output, ensuring safe and reliable operation of sensitive electronic components such as the ESP32, sensors, and communication modules.

(a)Camera Module *(b)Mini DVR*

FIG 3.9:Imaging Subsystem

In addition to power management, the subsystem also incorporates an imaging setup consisting of a compact camera module and a mini DVR (Digital Video Recorder). The camera module is responsible for capturing real-time video during the operation, which can be useful for monitoring flight conditions, payload deployment, or environmental observations. The mini DVR is interfaced with the camera module and is used to record the captured video onto a storage medium such as a microSD card. This allows for post-flight analysis and documentation of the mission.

The integration of the camera module with the mini DVR ensures that video data is recorded independently of the main control system, thereby reducing the processing load on the ESP32 and improving overall system efficiency. The entire imaging setup is powered through the regulated output of the UBEC, ensuring stable performance without voltage fluctuations. This combined power and imaging subsystem enhances the functionality of the system by providing both reliable power distribution and valuable visual feedback, making it an essential component of the overall design.

3.12 Navigation Module

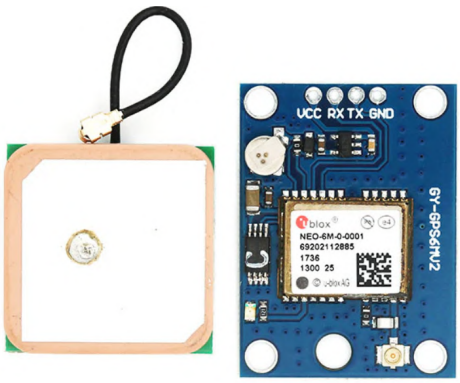


FIG 3.10:GPS Module

The navigation module is a critical component responsible for providing accurate real-time location information, including latitude, longitude, altitude, speed, and time. It operates by receiving signals from multiple satellites in the Global Positioning System (GPS) and determines position using trilateration techniques. This module is known for its high sensitivity, low power consumption, and reliable performance, making it highly suitable for embedded and aerospace applications.

The module communicates with the ESP32 microcontroller using UART (serial communication). It continuously transmits data in the form of NMEA sentences, which contain structured information about positioning and timing. The ESP32 processes these data strings to extract essential parameters, which can then be used for real-time tracking or transmitted to the ground station via LoRa communication.

A key advantage of the navigation module is its ability to maintain stable positioning even under dynamic conditions such as motion, vibration, and altitude variation. It also includes internal memory support and a backup battery, which helps retain satellite data and significantly reduces the time required for signal acquisition during startup.

In the overall system, this module plays a vital role in tracking and navigation. It enables precise monitoring of the system's position during operation and assists in recovery after landing. Additionally, the recorded positional data can be used for post-mission trajectory analysis, making it an essential element for both real-time monitoring and performance evaluation.

CHAPTER 4
BLOCK DIAGRAM

4.1 Overview

The block diagram of the Basilian-01 Rocket Avionics System represents the complete hardware architecture developed for the project. The system is designed to ensure reliable flight monitoring, long-range communication, autonomous recovery, and data logging during rocket operation. The architecture is divided into two main sections: the Ground Control System and the On-Board Rocket Avionics System. This separation allows real-time monitoring from the ground while enabling autonomous operation of critical flight functions onboard the rocket.The ground control system is responsible for receiving telemetry data and providing status feedback, whereas the onboard system performs all flight-critical operations such as sensor data acquisition, altitude estimation, data logging, and parachute deployment. Long-range RF communication is established between the two sections using LoRa technology operating in the 433 MHz band. Redundancy is incorporated in power regulation, flight controllers, and recovery mechanisms to enhance reliability and mission safety.

4.2 On-Board Power System

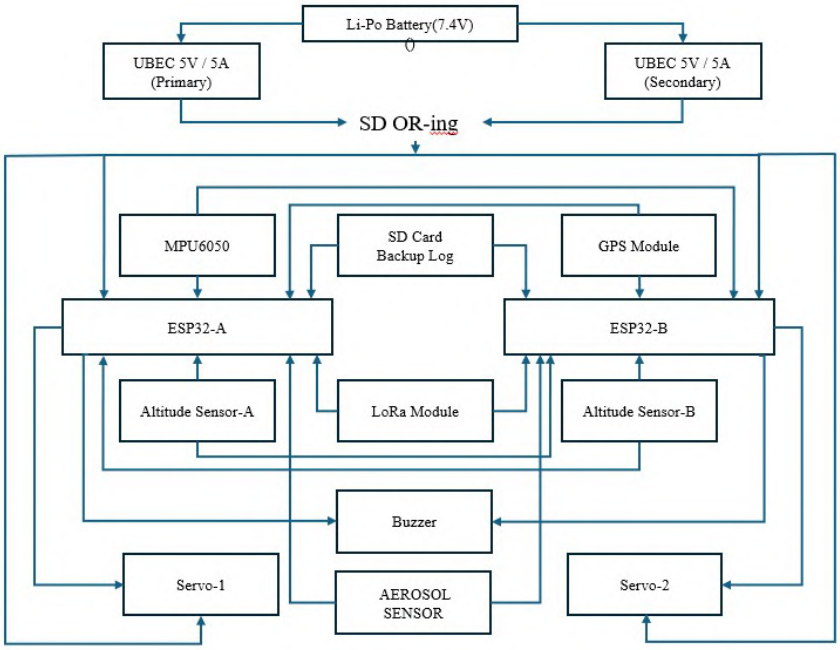


FIG 4.1:On Board

The onboard avionics system is powered by a dedicated Li-Po battery pack configured

for 2S or 3S operation. To ensure stable power delivery, two UBECs rated at 5 V, 3 A are used. These UBECs regulate the battery voltage and distribute power to the onboard electronics. The use of dual UBECs enhances power reliability and reduces the risk of system failure due to voltage drops or regulator malfunction.The regulated power output is supplied to the flight controllers, sensors, communication modules, data logging devices, and recovery actuators. This power architecture ensures uninterrupted operation of critical avionics throughout the flight. The onboard avionics system employs two ESP32 microcontrollers, designated as ESP32-A and ESP32-B, which function as primary flight controller units. These controllers operate in a redundant configuration to improve system fault tolerance. Each controller is capable of acquiring sensor data, executing flight logic, handling telemetry, and controlling recovery mechanisms.Redundancy in flight controllers ensures that critical operations such as flight monitoring and parachute deployment can still occur in the event of a controller failure. Both controllers communicate with onboard sensors and peripherals to maintain synchronized operation during flight.

4.3 Ground Control System

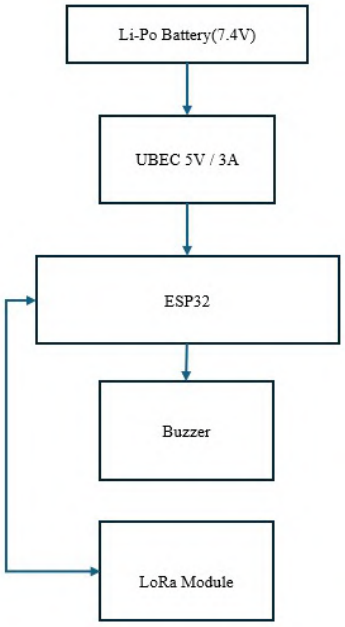


FIG 4.2:Ground Plane

The ground control system serves as the monitoring and communication interface between the rocket and the operator. It is powered using a Li-Po battery pack rated for 2S or 3S operation. The battery output is regulated by a UBEC, which provides a stable 5 V, 3 A supply required for the ground electronics.An ESP32-B microcontroller is used as the primary controller in the ground station. This controller is responsible for receiving telemetry data transmitted from the rocket, processing the received information, and

handling communication protocols. A LoRa SX1278 module operating at 433 MHz is interfaced with the ESP32-B to enable long-range RF communication. This allows the ground station to receive flight parameters such as altitude, system status, and deployment confirmation in real time.A buzzer module is integrated into the ground control system to provide audible feedback during transmission and reception events. This helps in quickly identifying communication activity and system status without requiring continuous visual monitoring. The ground control system does not perform any flight-critical decision-making and acts solely as a passive monitoring and communication unit.

4.4 Wireless Ignition

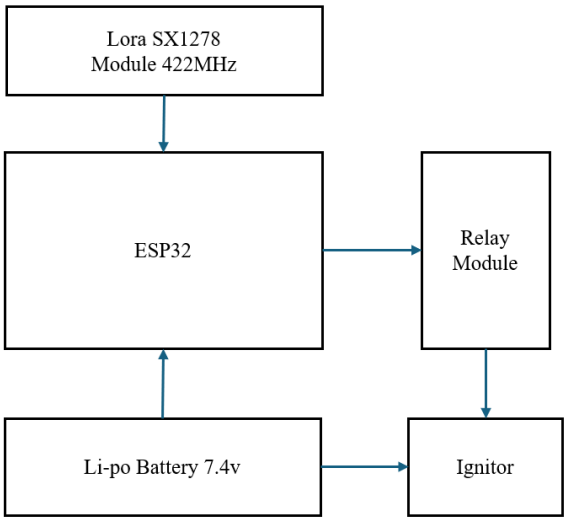


FIG 4.3:Wireless Ignition

The wireless ignition system operates through a coordinated interaction between the LoRa communication module, ESP32 microcontroller, relay module, power source, and ignitor. The LoRa SX1278 module, operating at 422 MHz, receives wireless commands from a remote transmitter and forwards them to the ESP32. Acting as the central control unit, the ESP32 processes the received signal and determines whether the ignition condition is met. Once validated, the ESP32 sends a control signal to the relay module, which functions as an electrically operated switch. The relay then completes the circuit between the Li-Po 7.4V battery and the ignitor. The battery provides the necessary power required for ignition, and when the relay is activated, current flows to the ignitor, initiating the ignition process. This architecture ensures safe and controlled wireless triggering, as the high-power ignition circuit is isolated from the low-power control circuitry.

4.5 Sensor and Data Acquisition System

The sensor subsystem is responsible for monitoring flight dynamics and environmental conditions. An MPU6050 accelerometer and gyroscope module is interfaced with

the primary flight controller to measure linear acceleration and angular velocity. This data is used to detect launch events, analyze flight behavior, and monitor motion stability.BME280 sensor modules are connected to both ESP32 controllers to measure temperature and pressure. The pressure data is used to estimate altitude, which is critical for flight phase detection and parachute deployment logic. The inclusion of duplicate sensors improves reliability and allows cross-verification of measurements.A GPS module is incorporated into the system to provide real-time latitude, longitude, and time information. This data supports flight tracking, post-flight recovery, and accurate timestamping of recorded flight data.

4.6 Communication and Data Logging System

To ensure guaranteed alert delivery during catastrophic events where internet connectivity may be disrupted, a GSM-based emergency messaging system is included as a fail-safe mechanism. When a disaster threshold is triggered, the gateway immediately sends SMS alerts to pre-registered responders containing details such as the detected hazard type, geographic node location, and severity estimate. This dual-alert mode combining GSM and cloud notifications significantly enhances the resilience of the system's communication structure. Even in worst-case disaster scenarios that damage network infrastructure, GSM networks typically remain functional longer, ensuring timely warnings reach authorities and communities at risk.

4.7 Recovery and Deployment System

The recovery system consists of two servo motors used for parachute deployment. Servo-1 acts as the primary deployment mechanism, while Servo-2 serves as a backup deployment system. These servos are controlled by the flight controllers based on altitude and flight phase detection.The inclusion of a backup servo enhances system reliability and reduces the risk of recovery failure due to mechanical or electrical faults. This dual-deployment strategy ensures safe descent and recovery of the rocket after flight.

CHAPTER 5
CORE SOFTWARE ARCHITECTURE

5.1 Introduction

The core software architecture of the Basilian-01 rocket forms the backbone of the entire avionics system, enabling real-time data acquisition, processing, decision-making, and communication throughout the mission. Unlike conventional embedded systems, rocket avionics software must operate under extreme and unpredictable conditions such as high vibration, rapid acceleration, temperature variations, and dynamic atmospheric pressure changes. These constraints demand a highly robust, deterministic, and fault-tolerant software design.

To address these challenges, the system follows a modular and layered architecture. Each functionality is divided into independent modules such as sensor interfacing, data processing, control logic, telemetry communication, and data logging. This modular approach enhances system reliability by isolating failures and simplifies debugging, testing, and future upgrades.

The architecture also emphasizes real-time execution. Time-critical operations such as flight state detection and deployment control are prioritized to ensure immediate response. The system is designed to operate continuously without interruption, ensuring that all mission-critical tasks are executed within strict timing constraints.

5.2 System Architecture Design

The Basilian-01 avionics software is divided into two primary segments: onboard flight software and ground station software. The onboard software is responsible for executing all mission-critical operations, while the ground station software provides visualization, monitoring, and post-flight analysis capabilities.

The onboard system utilizes a dual-controller architecture using ESP32 microcontrollers configured in an active-active redundancy mode. Both controllers independently perform all computations including sensor acquisition, altitude estimation, flight state determination, and deployment decision-making. This redundancy ensures high reliability, as the failure of one controller does not compromise the mission.

Task scheduling is implemented using a cooperative multitasking approach. Tasks are executed in predefined loops with specific time intervals. High-priority tasks such as sensor data acquisition and control logic are executed at higher frequencies, while lower-priority tasks such as telemetry transmission and data logging are scheduled less frequently. This ensures efficient CPU utilization and prevents system overload.

Inter-module communication is achieved through shared data structures and synchronized buffers. Care is taken to ensure data consistency and avoid race conditions. The

system is also designed to handle unexpected events through fallback mechanisms and safe state transitions.

Component	ESP32-A	ESP32-B	Function
Microcontroller	ESP32-WROOM-32	ESP32-WROOM-32	Flight control, decision logic
Barometric Sensor	BME280-A	BME280-B	Altitude measurement
IMU	MPU6050 (shared)	I^2C access	Accelerometer/gyroscope
GPS	NEO-6M (shared)	UART access	Position, timestamp
LoRa Transceiver	SX1278 (shared)	SPI access	Uplink to ground
Servo Actuator	Direct control	Direct control	Parachute deployment
SD Card Module	SPI access	SPI access	Data logging
Power	Battery + voltage divider	Battery + voltage divider	Power supply, monitoring

TAB 5.1:Components

5.3 Software Modules

The software architecture is divided into several key modules:

Sensor Interface Module: This module handles communication with all onboard sensors using protocols such as I2C, SPI, and UART. It is responsible for initializing sensors, acquiring raw data, and performing basic validation checks. The module ensures that all sensor readings are synchronized and available for further processing.

Data Processing Module: This module performs filtering, calibration, and computation of derived parameters. Techniques such as moving average filtering and noise reduction are applied to improve data accuracy. Parameters such as altitude, velocity, and rate of change are computed in real time.

Control Logic Module: This module implements the finite state machine (FSM) that governs the behavior of the rocket. It makes decisions based on sensor inputs and triggers actions such as deployment events.

Communication Module: Responsible for transmitting telemetry data using LoRa communication. It ensures reliable packet formation, encoding, and periodic transmission.

Data Logging Module: Stores all critical flight data onto an SD card. This ensures data availability for post-flight analysis even in case of communication failure.

The modular design ensures scalability, allowing additional features to be integrated without affecting existing functionalities.

5.4 Software Architecture Layers

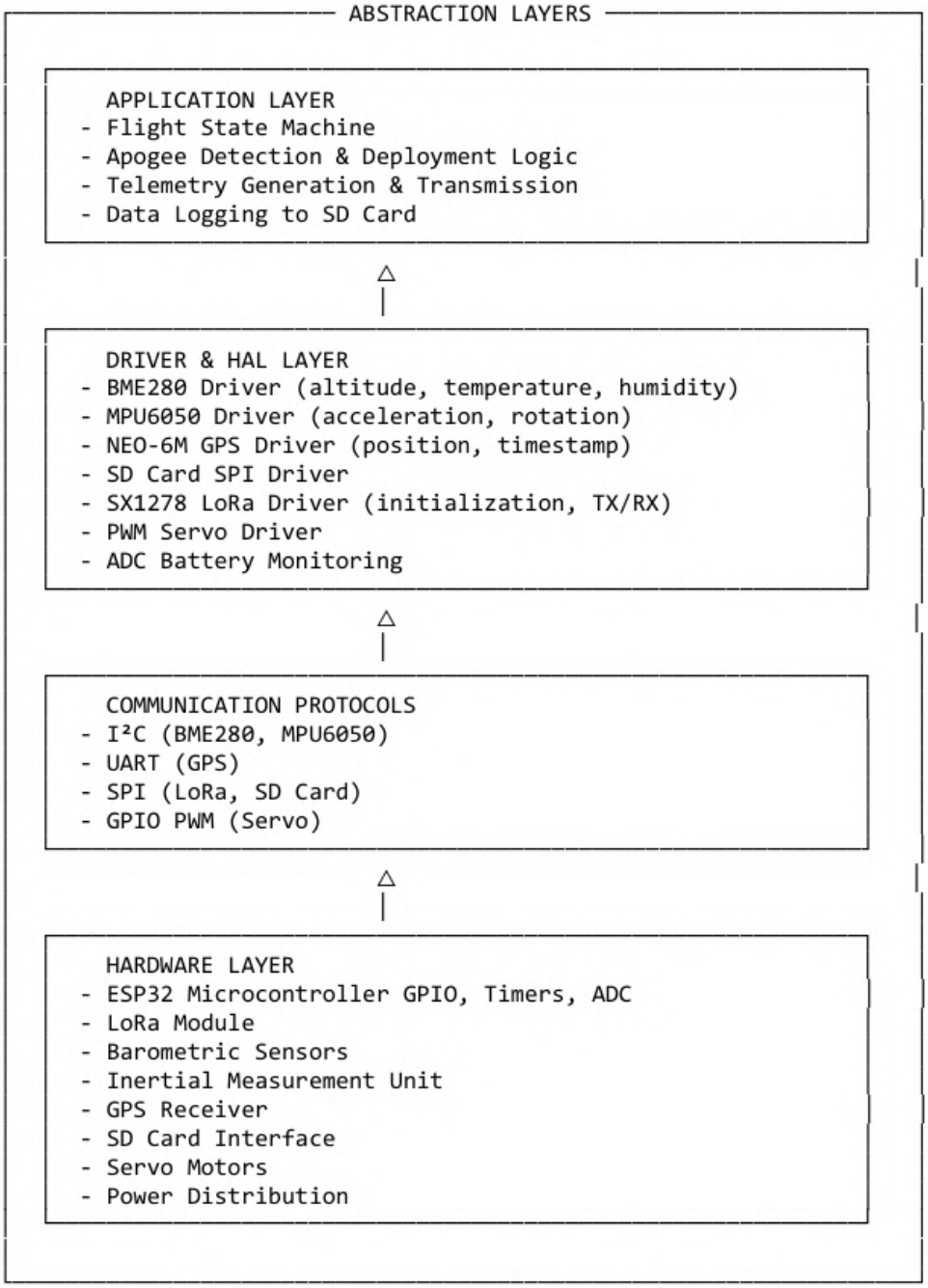


FIG 5.1:Architecture Layers

CHAPTER 6
FLIGHT STATE MACHINE DESIGN

6.1 Introduction

The Finite State Machine (FSM) is a fundamental component of the Basilian-01 avionics software, providing a structured and deterministic approach to managing the rocket's behavior throughout its flight. Due to the highly dynamic nature of rocket motion, it is essential to divide the entire flight into well-defined states, each representing a specific phase of operation. The FSM ensures that the system performs appropriate actions based on real-time sensor data and predefined conditions.

Unlike linear control systems, FSM-based design allows for clear separation of operational phases, reducing complexity and improving reliability. Each state has specific entry conditions, exit conditions, and associated actions, ensuring that transitions occur only when valid criteria are satisfied. This approach minimizes the risk of unintended behavior due to sensor noise or transient disturbances.

6.2 Definition of Flight States

The Basilian-01 rocket flight is divided into the following states:

Calibration State: This is the initial state where all sensors are initialized and calibrated. Baseline readings for pressure, temperature, and acceleration are established to ensure accurate measurements during flight.

Armed State: In this state, the system is prepared for launch. All systems are verified, and the rocket waits for launch conditions to be met.

Launch Detection State: Launch is detected when acceleration exceeds a predefined threshold, indicating liftoff.

Powered Ascent: During this phase, the rocket motor is active, and rapid altitude gain occurs. Continuous monitoring of acceleration and altitude is performed.

Coasting Phase: After motor burnout, the rocket continues to ascend due to inertia. Acceleration drops significantly, but altitude continues to increase.

Apogee State: This state represents the highest point of the flight trajectory. It is detected when altitude stops increasing and begins to decrease.

Descent Phase: Following deployment, the rocket descends under parachute control.

Landing State: This final state indicates that the rocket has reached the ground and all dynamic activities have ceased.

Abort State: This state indicates error and manual reset is provided.

6.3 State Transition Logic

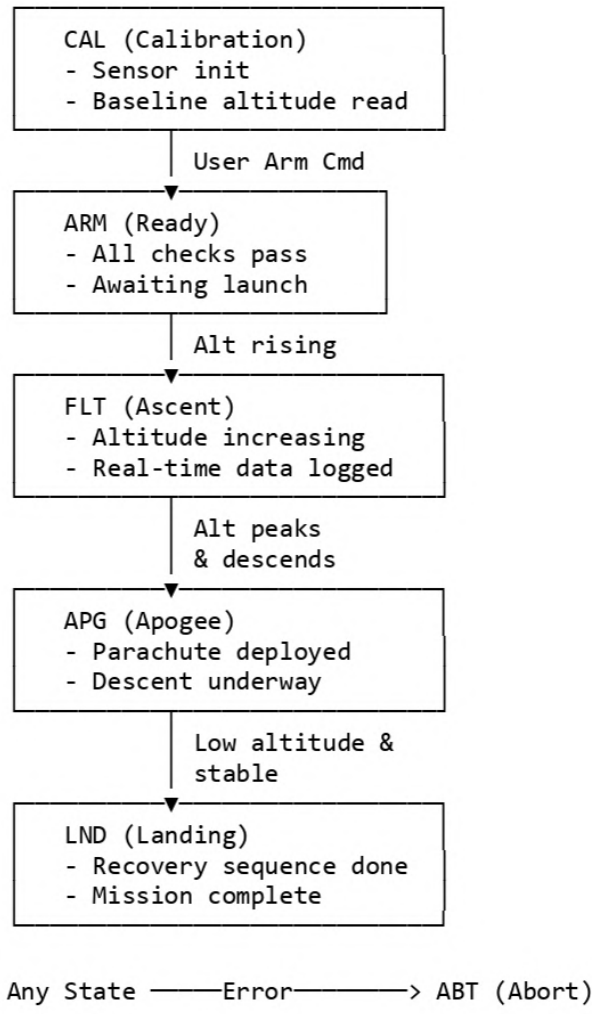


FIG 6.1:Transition Logic

State transitions are governed by carefully defined conditions based on sensor data. To prevent false triggers caused by noise or sudden disturbances, the system employs filtering techniques such as moving average and threshold validation across multiple samples.

For example, launch detection requires acceleration values to exceed a threshold consistently over several readings. Similarly, apogee detection is confirmed only after observing a continuous decrease in altitude over multiple samples. This multi-condition validation ensures robustness and minimizes erroneous transitions.

Additionally, timeout mechanisms are implemented to handle abnormal scenarios. If a state persists longer than expected, fallback transitions are triggered to maintain system safety.

6.4 Advantages of FSM Design

The FSM approach offers several advantages including predictability, ease of debugging, modularity, and scalability. It simplifies the implementation of complex control logic and ensures that system behavior remains well-defined under all operating conditions.

CHAPTER 7
REDUNDANT ALTITUDE ESTIMATION

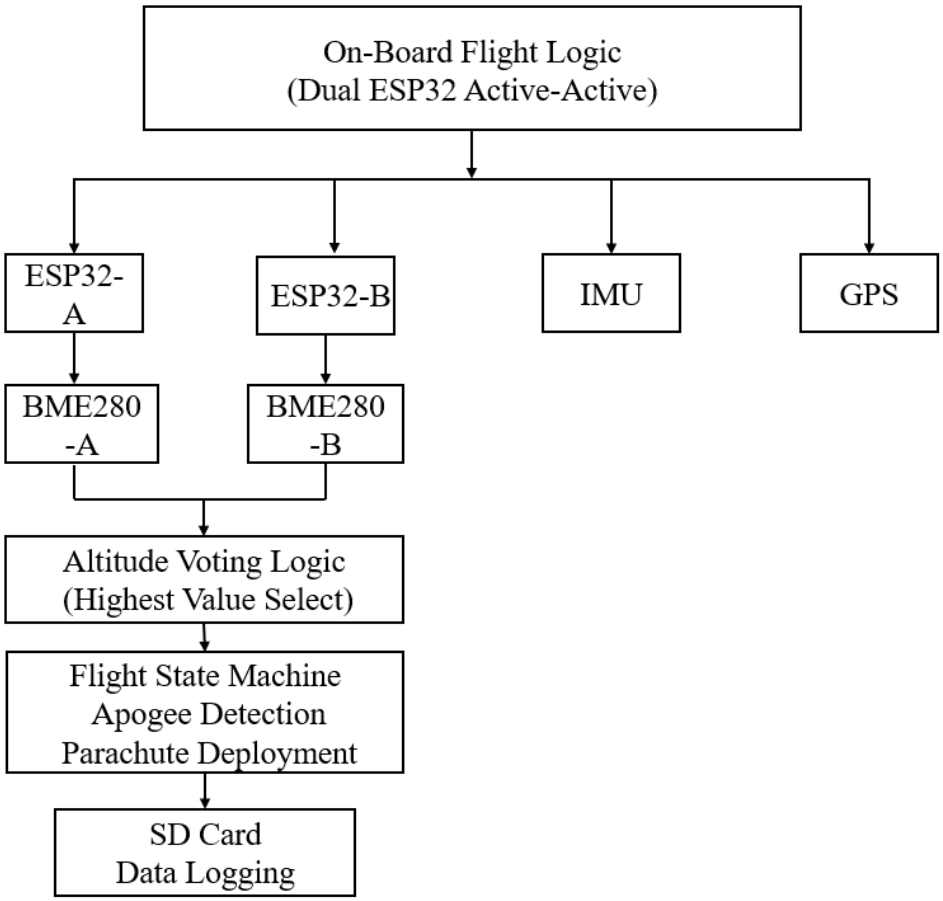


FIG 7.1:Redundancy

7.1 Introduction to Altitude Estimation

Altitude estimation is a fundamental requirement in rocket avionics, as it directly influences critical flight decisions such as apogee detection and parachute deployment. In the Basilian-01 system, altitude is continuously monitored throughout the flight to track ascent, identify peak altitude, and control descent mechanisms.

Unlike GPS-based altitude estimation, which may suffer from latency and signal loss at high speeds or altitudes, barometric sensing provides a more reliable and responsive method. The system is designed to process altitude data in real time, ensuring that even rapid changes in flight conditions are accurately captured. This capability is essential in dynamic environments where delays or inaccuracies can lead to mission failure.

Furthermore, altitude estimation is not a standalone process but is integrated with other subsystems such as the flight state machine and deployment logic. Therefore,

maintaining high accuracy and reliability in altitude measurement is critical for overall system performance.

7.2 Barometric Altitude Calculation

The Basilian-01 system calculates altitude using barometric pressure readings obtained from onboard sensors such as the BME280. The relationship between atmospheric pressure and altitude is governed by standard atmospheric equations, where pressure decreases exponentially with increasing altitude.

The system converts raw pressure readings into altitude values using calibrated mathematical models. These models take into account factors such as temperature and baseline pressure at ground level. During initialization, a reference pressure value is recorded, which serves as a baseline for all subsequent calculations.

The altitude calculation process involves continuous sampling of pressure data, followed by conversion using the atmospheric model. This ensures that altitude is updated in real time with high precision. Special care is taken to handle rapid pressure changes during ascent, ensuring that the computed altitude remains stable and accurate.

7.3 Noise Filtering and Calibration

Sensor readings in rocket environments are highly susceptible to noise due to vibration, turbulence, and electrical interference. To address this issue, the Basilian-01 system incorporates advanced filtering and calibration techniques.

Filtering is performed using methods such as moving average filtering and exponential smoothing. These techniques help eliminate sudden spikes and fluctuations in sensor readings, resulting in smoother altitude data. The choice of filter parameters is carefully optimized to balance responsiveness and noise reduction.

Calibration is performed during the initial phase of operation. The system records baseline values for pressure and temperature when the rocket is stationary. These values are used to correct subsequent readings, ensuring consistency and accuracy throughout the flight.

Additionally, the system continuously monitors sensor performance and applies adaptive correction techniques if deviations are detected. This dynamic calibration approach enhances measurement reliability under varying environmental conditions.

7.4 Redundancy Implementation

To ensure reliability, the Basilian-01 system employs redundancy in altitude estimation through both hardware and software mechanisms. Dual sensors and dual microcontrollers are used to independently measure and compute altitude.

Each microcontroller processes data from its respective sensor and calculates altitude

independently. The results are then compared to identify discrepancies. If the difference between the two values exceeds a predefined threshold, the system flags an inconsistency and applies corrective measures.

Redundancy also extends to data validation. Multiple samples are analyzed before confirming critical conditions such as apogee detection. This multi-layered redundancy significantly reduces the risk of failure due to sensor malfunction or computational errors.

7.5 Importance of Redundancy

Redundancy is essential in aerospace systems where failure can have severe consequences. By incorporating redundant altitude estimation, the Basilian-01 system ensures continuous operation even in the presence of faults.

The use of redundancy enhances fault tolerance, improves data reliability, and increases mission success probability. It also provides a mechanism for error detection and correction, allowing the system to maintain accurate altitude tracking under all conditions.

CHAPTER 8
APOGEE DETECTION AND DEPLOYMENT LOGIC

8.1 Importance of Apogee Detection

Apogee detection represents one of the most critical functional requirements in the Basilian-01 avionics system. Apogee is defined as the maximum altitude reached by the rocket during its flight trajectory. Accurate identification of this point is essential for initiating the recovery sequence, particularly the deployment of the parachute system.

If apogee is detected too early, the parachute may deploy during ascent, leading to excessive drag forces and potential structural damage. Conversely, delayed detection can result in deployment at high descent velocities, increasing the risk of impact damage. Therefore, precise and reliable apogee detection is essential for mission success.

In dynamic flight environments, sensor noise, atmospheric turbulence, and mechanical vibrations can affect altitude readings. Hence, the detection system must be robust enough to differentiate between actual altitude trends and transient fluctuations. The system is designed to operate with high reliability even under such adverse conditions.

8.2 Apogee Detection Algorithm

The apogee detection algorithm in Basilian-01 is based on real-time monitoring of altitude trends derived from barometric pressure data. The fundamental principle is that altitude increases during ascent and decreases after reaching apogee.

To ensure reliability, the algorithm does not rely on a single measurement. Instead, it analyzes a sequence of altitude readings over time. A sliding window approach is used, where multiple consecutive samples are evaluated. Apogee is confirmed only when a consistent decreasing trend is observed across several readings.

Mathematically, the algorithm computes the difference between successive altitude samples. If the differences remain negative for a predefined number of cycles, the system confirms apogee. Additionally, filtering techniques such as moving averages are applied to reduce noise and improve stability.

The algorithm also includes threshold constraints to prevent false triggering due to minor fluctuations. This multi-layer validation approach significantly enhances detection accuracy.

8.3 Deployment Mechanism

Upon confirmation of apogee, the deployment mechanism is activated to release the parachute. The Basilian-01 system employs a servo-based mechanical deployment system

controlled via PWM signals generated by the ESP32 microcontroller.

The servo motor is pre-calibrated to rotate to a specific angle corresponding to the release position. The control signal is generated with precise timing to ensure smooth and reliable actuation. The system also incorporates delays and stabilization intervals to prevent abrupt movements that could affect the rocket's structural integrity.

Mechanical considerations such as torque requirements, mounting stability, and vibration resistance are taken into account during design. The deployment mechanism is tested under simulated conditions to ensure reliability during actual flight.

8.4 Redundant Deployment Logic

To enhance mission safety, redundant deployment strategies are incorporated alongside the primary apogee-based trigger. These backup mechanisms ensure that the parachute is deployed even if the primary detection algorithm fails.

One such method is time-based triggering, where deployment is initiated after a predefined duration from launch. Another method involves secondary altitude thresholds, where deployment occurs if the rocket descends below a certain altitude without prior activation.

Additionally, both microcontrollers independently execute deployment logic. If either controller detects apogee or meets backup conditions, it can trigger deployment. This dual-trigger system significantly improves reliability and reduces the probability of failure.

CHAPTER 9
DERATED ANALYSIS

9.1 Consolidated Table (Voltage, Typical/Max Current, Interface Frequency)

Device	Operating Voltage	Typical current	Frequency
MPU6050	**2.375–3.46 V**	Gyro/Accel full power ≈ **3.6–3.8 mA**	I²C up to **400 kHz** (internal ODR: accel ≤1 kHz, gyro ≤8 kHz)
BME280	VDD **1.71–3.6 V**	**3 uA** to **600 uA** at **3.4MHz**	I²C up to **3.4 MHz**
LoRa(SX127x)	**3.3 V** (VDD & I/O)	RX = **10–20 mA** TX ≈ **20–80 mA**	**433 MHz**
Servo	**5.0 V** (typical)	**10–100 mA** depending on size	PWM from ESP32 GPIO (**50 Hz** typical; pulse width 1–2 ms)
GPS NEO-6M	**2.7 → 3.6 V**	**45–50 mA** under normal operation	**1Hz** to **5Hz**
ESP32(what it can provide	Module: **3.0–3.6 V**	Peak current **260 mA**/ individual GPIO max = **40 mA**	I²C master stable: **100/400 kHz**

TAB 9.1:Consolidated table

9.2 Practical derating guidelines (numbers & percentages)

• Voltage derating: Operate at standard 3.3 V for 3.3 V devices (MPU6050, BME280, LoRa). This is comfortably below device max (e.g., BME280 max 3.6 V, MPU6050 3.46 V). Treat 3.3 V as 90–95% of absolute max — no further reduction usually possible for logic rails.

• Regulator current derating: Size regulator for expected peaks + margin, then use 80% of regulator continuous rating for sustained loads. (Example: a 2 A regulator → treat 1.6 A continuous usable.)

• GPIO derating: Absolute GPIO max 40 mA → derate to 50% → 20 mA continuous. Do not power peripherals from GPIO.

• LoRa derating: If module can TX at +17 dBm (100–120 mA), derate to 150–200

mA budget to cover higher TX power, temperature derating, and inefficiencies. If you plan repeated long TXs at high power, use a separate power supply/regulator or ensure regulator is rated for sustained high current (derated to 80%).

• Servo sizing: Always size servo supply by sum of expected stall/peak currents and allow 100–300% headroom per servo for worst-case starts/stalls. Use separate 5 V UBEC/ESC rated for continuous current (N_servos × typical_operating_current × 1.5) and ensure it can handle short peaks; add local bulk caps (electrolytic 470–2200 µF depending on number/size).

9.3 Percentages Table

Device	Vop (V)	Vmax (V)	Voltage usage % = Vop/Vmax	Voltage derate margin % = 1−(Vop/Vmax)	Ityp (mA)	% of ESP peak (260 mA)	% of derated regulator (1.2 A)
BME280	3.3	3.6	91.67%	8.33%	0.0036	0.0014%	0.0003%
MPU6050	3.3	3.46	95.32%	4.68%	3.8	1.46%	0.32%
LoRa (SX127x)	3.3	3.6	91.67%	8.33%	100	38.46%	8.33%
NEO-6M GPS	3.3	3.6	91.67%	8.33%	50	19.23%	4.17%
Servo (hobby)	5.0	6.0	83.33%	16.67%	100 (typ)[1]	38.46%	8.33%

TAB 9.2:Percentage Table

9.4 Assumptions / References Used

• Chosen operating voltage for 3.3 V devices: Vop = 3.3 V.

• Chosen operating voltage for servo: Vop = 5.0 V (typical servo supply).

• Absolute max voltages (datasheet typical maxima):

o BME280 Vmax = 3.6 V

o MPU6050 Vmax = 3.46 V

o LoRa (SX127x) Vmax = 3.6 V (module typical)

o NEO-6M core Vmax = 3.6 V (module often has onboard regulator)

o Servo Vmax 6.0 V (typical hobby servo recommended max)

• Typical currents used:

o BME280: 3.6 µA = 0.0036 mA (1 Hz full sensors typical)

o MPU6050: 3.8 mA (gyro + accel full power typical)

o LoRa (typical active TX): 100 mA (use 100 mA as a typical TX draw; module bursts can be higher)

o Servo (typical moving): 100 mA (note: stall/start peaks can be 1 A — see note)
o NEO-6M GPS module: 50 mA (typical tracking)
• ESP reference currents:
o ESP32 peak TX baseline = 260 mA (used earlier)
o 3.3 V regulator rated 1.5 A, derated to 80% → 1.2 A usable continuous

9.5 Formulas Used

• Voltage usage % = (Vop / Vmax) × 100%
• Voltage derating margin = (1 − Vop / Vmax) × 100% — how much headroom you have below absolute max.
• Current % of ESP peak = (Ityp / 260 mA) × 100%
• Current % of derated regulator = (Ityp / 1,200 mA) × 100%

9.6 Conclusion

Based on the applied derating considerations, all components in the system operate within their specified limits with sufficient safety margins. The voltage levels, current capacity, and power handling are maintained below maximum ratings, ensuring reliable and stable performance. Therefore, the selected components are expected to function properly under the given specifications.

CHAPTER 10
TELEMETRY AND DATA LOGGING SYSTEM

10.1 Telemetry System Design

The telemetry system is responsible for transmitting real-time flight data from the rocket to the ground station. This enables operators to monitor system performance and track flight parameters during the mission.

The Basilian-01 system utilizes LoRa communication technology due to its long-range capabilities and low power consumption. Operating in the 433 MHz frequency band, LoRa provides reliable communication over several kilometers under line-of-sight conditions.

The telemetry system is designed to balance data transmission rate and power efficiency. Since rocket missions prioritize reliability over bandwidth, only essential parameters are transmitted at optimized intervals. This ensures consistent communication without overloading the system.

10.2 Data Packet Structure

The structure of telemetry data packets is carefully designed to ensure efficient and reliable transmission. Each packet contains a combination of sensor data, system status information, and error-checking fields.

Typical parameters included in the packet are altitude, acceleration, temperature, pressure, GPS coordinates, battery voltage, and system state. Each parameter is encoded in a compact format to minimize packet size.

A header is included to identify the start of the packet, followed by payload data and a checksum for error detection. The use of structured packets ensures that the ground station can accurately decode and interpret the received data.

10.3 Error Handling

Reliable communication requires robust error handling mechanisms. The Basilian-01 telemetry system incorporates techniques such as checksums and cyclic redundancy checks (CRC) to detect errors in transmitted data.

If a packet is found to be corrupted, it is discarded to prevent incorrect data interpretation. In addition, redundancy in transmission may be implemented by periodically sending critical parameters multiple times.

The system is also designed to handle temporary communication loss. In such cases, data logging ensures that no information is permanently lost, allowing for post-flight recovery and analysis.

10.4 Data Logging

Data logging is implemented using an onboard SD card module. All flight data is recorded in real time in a structured format such as CSV or binary logs. This provides a complete record of the mission for analysis and debugging.

The logging system is optimized to handle high-frequency data without causing delays in other critical tasks. Buffering techniques are used to store data temporarily before writing to the storage medium.

In addition to serving as a backup for telemetry, data logging enables detailed post-flight analysis, helping improve system performance and design in future missions.

10.5 Telemetry Packet Format

The on-board system transmits CSV-formatted telemetry packets at regular intervals (typically every 50–100 ms during flight):
STATE,temp,hum,press,altCm,battPct,pitch,roll,yaw,ax,ay,az,gx,gy,gz,lat,lon,g psFix,sats, hdop,speed kmph,timestamp,RSSI.
Example Packet: FLT,25.3,45.2,987.6,1250,87.4,12.5,-8.3,22.1,0.2,0.8,-9.4,0.1,-0.3,0.2,13.054 1,80.2384,1,8,1.2,15.3,1704067523,−110

Field	Type	Unit	Description
STATE	String	—	Flight state (CAL, ARM, FLT, APG, LND, ABT)
temp	Float	°C	Temperature from BME280
hum	Float	% RH	Relative humidity from BME280
press	Float	hPa	Atmospheric pressure from BME280
altCm	Float	cm	Effective altitude (max of dual sensors)
battPct	Float	%	Battery charge level
pitch, roll, yaw	Float	degrees	Orientation from IMU
ax, ay, az	Float	m/s^2	Linear acceleration from IMU
gx, gy, gz	Float	rad/s	Angular velocity from IMU
lat, lon	Float	degrees	GPS position
gpsFix	Int	—	GPS fix quality (0=none, 1=2D, 2=3D)
sats	Int	count	Number of satellites tracked
hdop	Float	—	Horizontal dilution of precision
speed_kmph	Float	km/h	Ground speed from GPS
timestamp	Long	Unix epoch (IST)	GPS-derived time (IST timezone)
RSSI	Int	dBm	Received signal strength (ground measurement)

TAB 10.1:Telemetry Packets

10.6 Command transmission

Command	Format	Purpose
PING	`PING`	Link verification (no echo)
PING with Token	`PING,<token>`	Link test with ACK-based acknowledgment
Release Parachute	`RELEASE`	Manual parachute deployment command (test only)
Hold Servo	`HOLD`	Lock servo in current position
Servo Angle Control	`angle1,angle2`	Manual servo angle setting (degrees, 0–180)
System Reset	`RESET`	On-board system restart

TAB 10.2:Command Transmission

CHAPTER 11
GROUND STATION SOFTWARE

11.1 Overview

The ground station software serves as the primary interface between the rocket and the operator. It receives telemetry data transmitted from the rocket and presents it in a structured and user-friendly format.

The software is typically implemented on a computer or embedded system connected to a LoRa receiver. It continuously listens for incoming data packets and processes them in real time. The processed data is displayed on a graphical user interface (GUI), allowing operators to monitor flight conditions.

11.2 Features

The ground station software includes several features designed to enhance usability and situational awareness. Real-time graphs display parameters such as altitude, velocity, and temperature, enabling visual analysis of flight behavior.

Numerical displays provide precise values for critical parameters, while status indicators show the current state of the rocket. Alert mechanisms are incorporated to notify operators of abnormal conditions such as sensor failure or unexpected altitude changes.

The interface is designed to be intuitive and responsive, ensuring that operators can quickly interpret data during flight.

11.3 Data Analysis

After the mission, the recorded telemetry data is used for detailed analysis. The ground station software supports data export and visualization tools that allow engineers to evaluate system performance.

Graphical analysis helps identify trends such as ascent rate, apogee accuracy, and descent characteristics. Statistical methods can be applied to assess sensor performance and detect anomalies.

This analysis plays a crucial role in improving future designs, optimizing algorithms, and enhancing overall system reliability.

CHAPTER 12
SOFTWARE SAFETY AND REDUNDANCY

12.1 Importance of Safety

Safety is a critical aspect of rocket avionics systems, as failures can lead to loss of equipment and potential hazards. The Basilian-01 software is designed with multiple safety layers to ensure reliable operation under all conditions.

The system must handle unexpected scenarios such as sensor failure, communication loss, and power fluctuations. Therefore, robust safety mechanisms are integrated into the software architecture to detect and mitigate such issues.

12.2 Redundancy Mechanisms

Redundancy is implemented using dual microcontrollers operating in an active-active configuration. Both controllers perform identical computations and monitor system parameters independently.

This approach ensures that if one controller fails, the other continues to operate without interruption. Redundant sensors and communication channels further enhance system reliability.

Function	Redundancy Type	Failure Mode Handling
Flight Computer	Dual ESP32 (independent)	Either controller can trigger recovery
Altitude Sensing	Dual BME280 + voting logic	Single sensor failure tolerated via max selection
Parachute Actuation	Dual servo control	Either servo line failure handled by remaining servo
Data Logging	On-board SD card	Ground link loss does not affect logging
Power Supply	Single battery with monitoring	Voltage divider reports battery state
Failsafe Recovery	Automatic apogee logic	No ground command required

TAB 12.1:Redundancy Mechanisms

12.3 Fault Detection

Fault detection mechanisms include watchdog timers, sensor validation checks, and communication monitoring. Watchdog timers reset the system in case of software malfunction, preventing system freeze.

Sensor validation ensures that readings remain within acceptable ranges. If abnormal values are detected, the system flags an error and switches to backup logic.

12.4 Fail-safe Operation

Fail-safe mechanisms ensure that critical operations such as parachute deployment are executed even in the presence of faults. Backup triggers and redundant logic paths guarantee that essential functions are not compromised.

The system is designed to transition into safe states during abnormal conditions, minimizing risk and ensuring controlled operation throughout the mission.

12.5 Single-Point Failure

Component	Failure	Impact	Mitigation
ESP32-A	Crash or reboot	ESP32-B maintains flight control and can deploy parachute	Active-active redundancy
ESP32-B	Crash or reboot	ESP32-A maintains flight control and can deploy parachute	Active-active redundancy
BME280-A	Sensor malfunction	Uses altitude from BME280-B; max selection ensures peak capture	Dual altitude voting
BME280-B	Sensor malfunction	Uses altitude from BME280-A; max selection ensures peak capture	Dual altitude voting
Servo Line	Cable cut or electrical break	Remaining servo can still deploy parachute	Dual servo actuation
LoRa Link	Complete loss	Flight continues, apogee detection and deployment happen on-board	On-board autonomy
SD Card Failure	Unable to write	Flight control unaffected (logging is non-blocking)	Graceful degradation
GPS Loss	No fix achieved	Flight control unaffected (IMU/baro provide state)	GPS is supplementary

TAB 12.2:Single Point Failure

CHAPTER 13
WEBSITE

13.1 Dashboard Overview

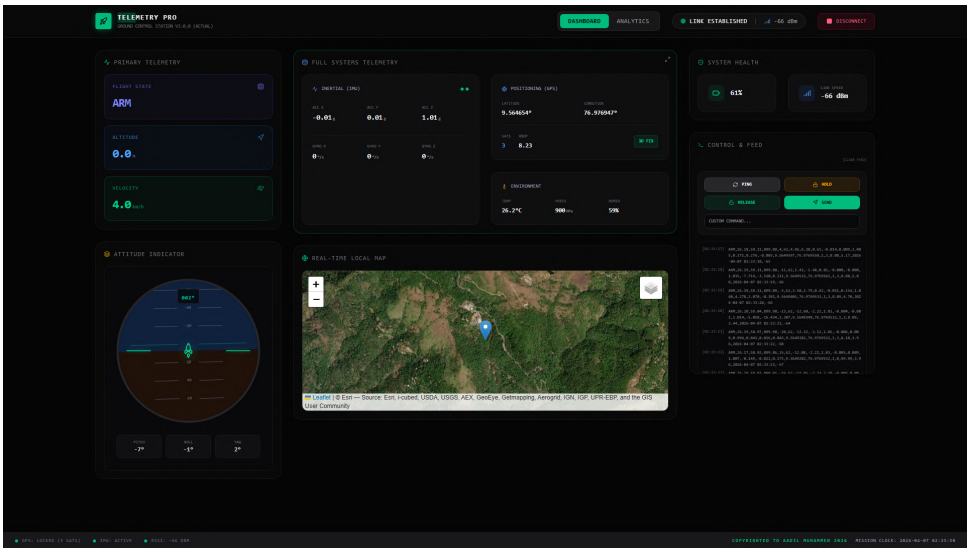


FIG 13.1: Dashboard Overview

The Dashboard Overview serves as the unified command center for the Ground Control Officer. It aggregates all critical mission parameters onto a single pane of glass—combining flight state, inertials, GPS mapping, command consoles, and attitude visualization. This ensures situational awareness is maintained instantly without navigating through multiple tabs during a fast-paced launch sequence.

13.2 Primary Telemetry

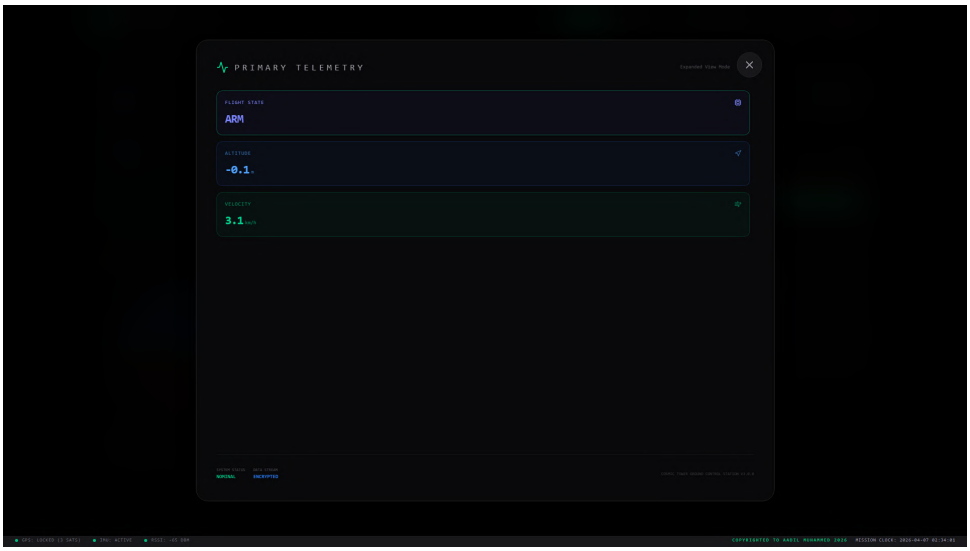


FIG 13.2: Primary Telemetry

The Primary Telemetry module isolates the most critical flight data required to confirm a successful launch. It displays the active Finite State Machine (FSM) status (e.g., Armed, Powered Ascent, Apogee, Descent), the real-time Barometric Altitude, and the calculated descent or ascent Velocity. This gives immediate visual confirmation of the rocket's current trajectory phase.

13.3 Full Systems Telemetry

FIG 13.3: Full Systems Telemetry

The Full Systems Telemetry view provides deep diagnostic insight into the hardware's performance. It features live, dynamically scaling line charts plotting the raw Accelerometer and Gyroscope axes to monitor vibration and stability. It also tracks historical environmental conditions (Temperature, Pressure, Humidity) and active satellite positioning coordinates in real-time.

13.4 System Health

FIG 13.4: System Health

The System Health module is vital for preventing mid-flight logic failures. It actively monitors the internal battery percentage of the onboard ESP32 flight controller to ensure sufficient voltage remains to fire the parachute servos. Additionally, it charts the LoRa RSSI (Received Signal Strength Indicator), allowing operators to predict line-of-sight dropouts or telemetry blackouts.

13.5 Control and Feed

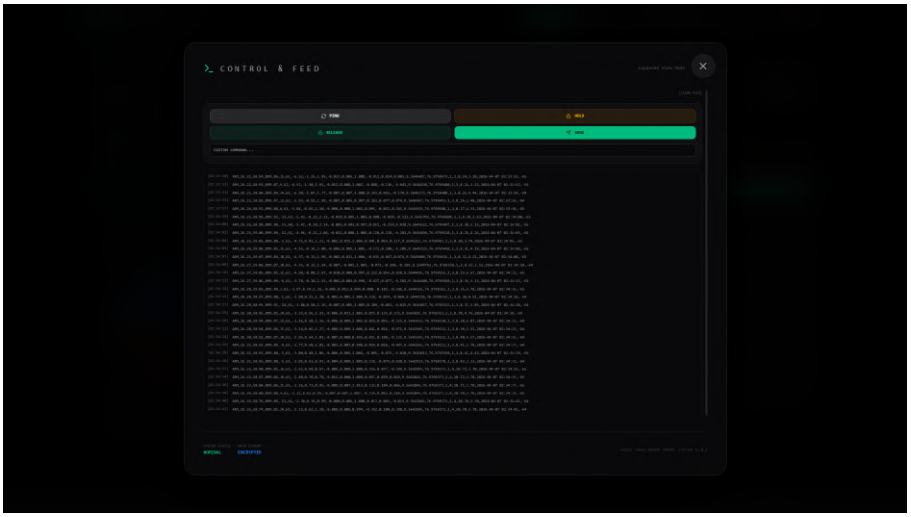


FIG 13.5: Control and Feed

The Control & Feed panel bridges remote manual operations and raw data debugging. It provides hardware command buttons (Ping, Hold, Release, Send) that broadcast over the 433MHz frequency to the physical rocket. Below the controls, a scrolling terminal displays the raw, unprocessed CSV packets exactly as they are received by the LoRa ground station, aiding in real-time troubleshooting.

13.6 Map Tracking

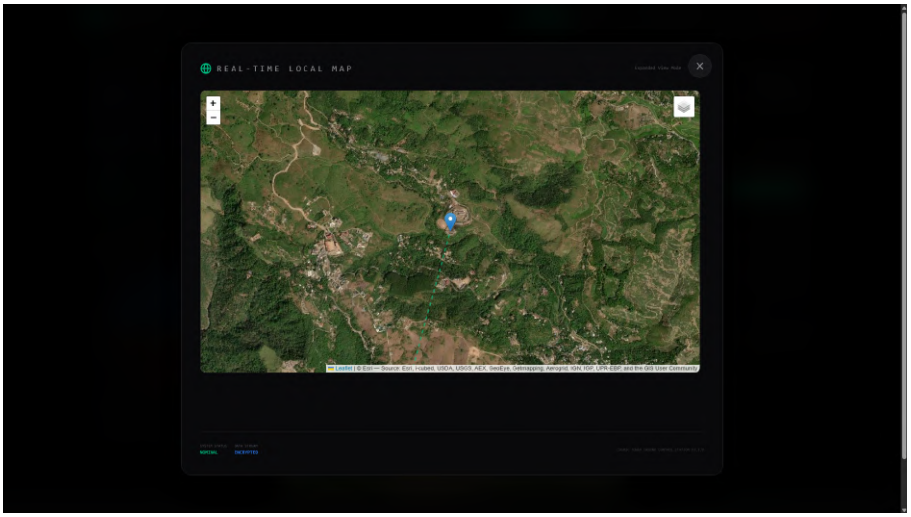


FIG 13.6: Map Tracking

The Map Tracking interface integrates Leaflet mapping layers to transpose the NEO-6M GPS coordinates directly onto 2D topography. This provides a live trajectory of the flight vehicle and establishes a dynamic recovery target radius, enabling ground teams to visually track and physically recover the payload after a parachute descent.

13.7 Attitude Indicator

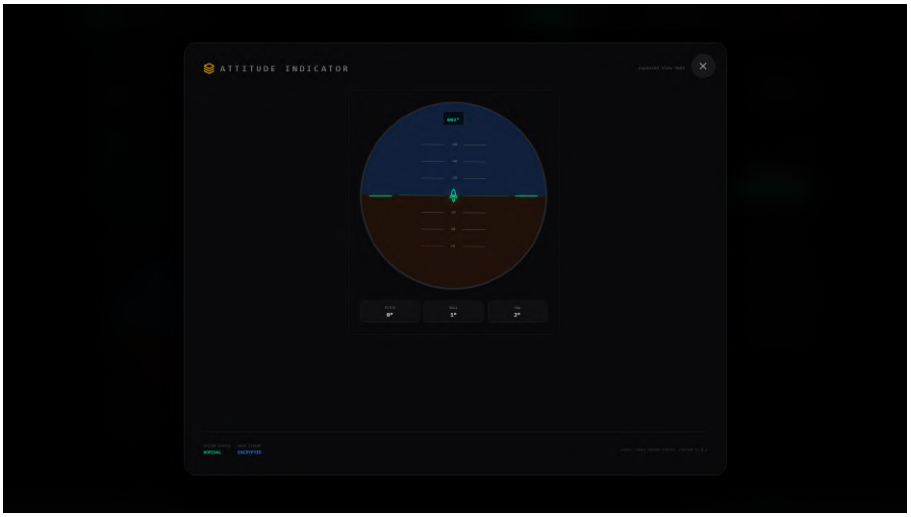


FIG 13.7: Attitude Indicator

The Attitude Indicator abstracts complex quaternion and Euler angle math from the MPU6050 into an easy-to-read aircraft-style digital horizon. It instantly visually communicates whether the rocket has pitched over, induced a sudden roll, or deviated off its parallel flight path without requiring operators to interpret raw numbers.

13.8 Analytics Overview

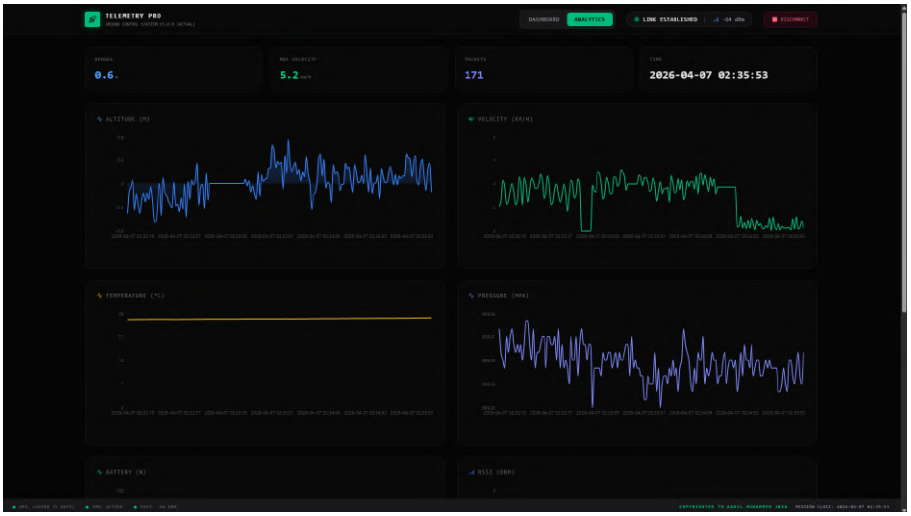


FIG 13.8: Analytics Overview

The Analytics Overview tab is dedicated to post-flight and in-flight technical review. It showcases interactive line charts tracking Altitude, Velocity, Temperature, and Pressure against the mission clock. It clearly visualizes the peak of the arc, proving mathematically exactly when and where Apogee was detected and the parachutes were deployed.

13.9 Recent Packet Logs

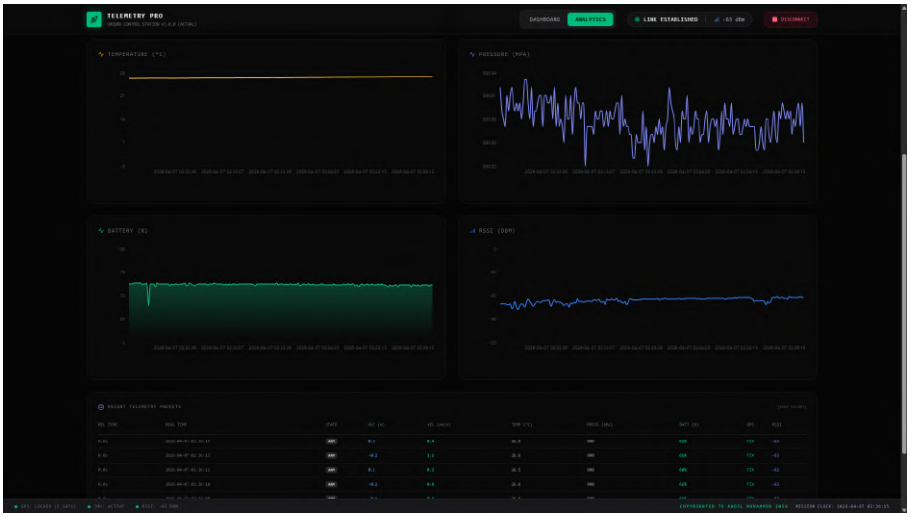


FIG 13.9: Recent Packet Logs

The Recent Packet Logs section provides a structured, tabular audit trail of the PostgreSQL database. It consolidates the high-frequency telemetry into human-readable rows, freezing exact timestamps and parameter states for meticulous post-mission analysis and safety auditing.

13.10 Launch Sequence Console

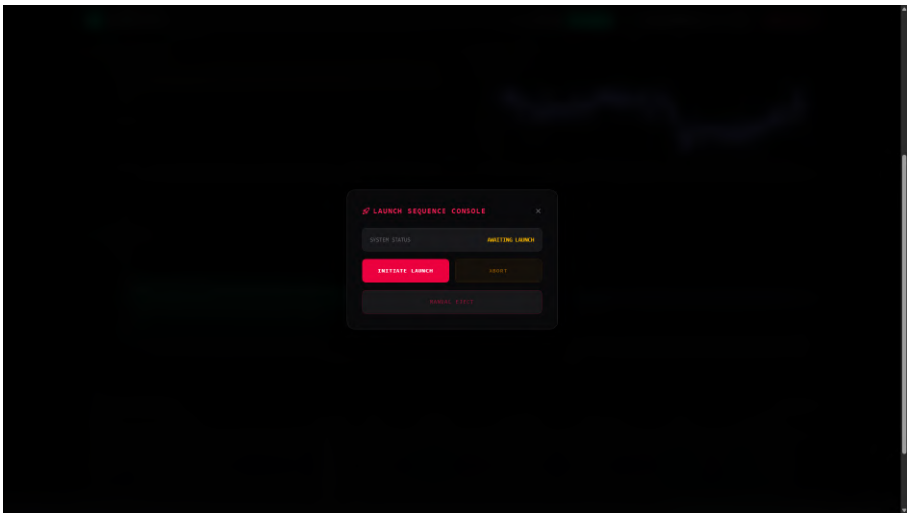


FIG 13.10: Launch Sequence Console

The Launch Sequence Console is a high-stakes, secure modal overlay. It specifically prevents accidental ground firings by explicitly requiring the operator to authorize and transmit the ”Initiate Launch” command. Once authorized, this digitally triggers the distant SCUB Ignition Module's relay, latching the 7.4V battery array directly to the ignitor for liftoff.

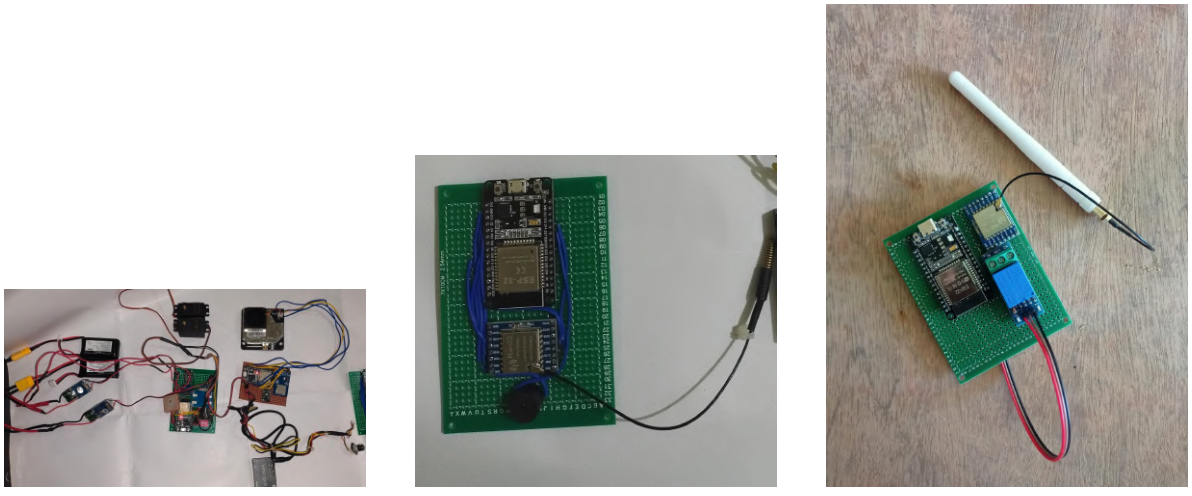
CHAPTER 14
RESULTS

As a result of this project, the Basilian-01 rocket avionics system was successfully developed and tested for real-time data acquisition, processing, and communication. The system integrates multiple sensors, onboard control logic, wireless telemetry, and ignition mechanisms to ensure reliable mission execution. The onboard computer continuously monitors parameters such as altitude, pressure, and environmental conditions, and processes them to determine the flight state of the rocket.

The wireless communication system effectively transmits telemetry data from the rocket to the ground station, enabling real-time monitoring and analysis. The ground station interface displays critical flight parameters, allowing operators to observe system behavior throughout the mission. The integration of LoRa communication ensures long-range data transmission with minimal loss.

The wireless ignition system was also tested and demonstrated reliable triggering capability from the ground station. The ignition signal is transmitted securely and activates the onboard ignition circuit, ensuring proper initiation of the launch sequence. The system design ensures safe operation with proper isolation between control and power circuits.

The onboard data logging system successfully records all critical flight parameters onto storage devices for post-flight analysis. This ensures that even in the event of communication failure, the data can be retrieved and analyzed later.



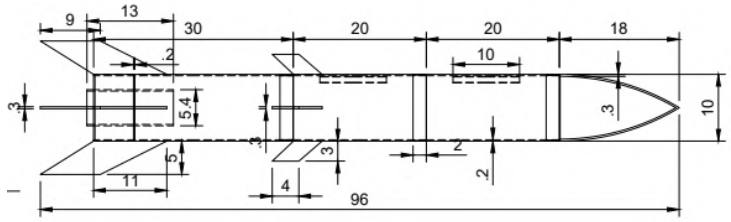
(a)On-Board *(b)Ground Plane* *(c)Wireless Ignition*

FIG 14.1:Circuit Boards

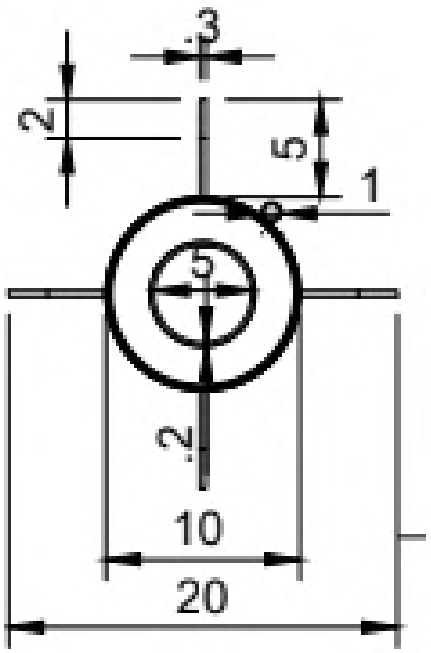
Although the system performs effectively, certain limitations were observed. Communication delays may occur under long-range or interference conditions, and sensor

readings may be slightly affected by environmental noise and dynamic flight conditions. However, the overall system demonstrates reliable performance and validates the feasibility of implementing a low-cost, modular, and scalable rocket avionics system.

The results confirm that the designed system is capable of performing all intended functions, including data acquisition, processing, communication, ignition control, and data logging, thereby ensuring successful mission operation.



(a)Front View



(b)Top View

FIG 14.2:System Dimensions

CHAPTER 15

ACHIEVEMENTS & ARTICLES

Sibi B John

Beval Philip Mathew

Aadil Muhammed

Rahul B

FIG 15.1 :Certificates

വിദ്യാർഥികൾ നിർമിച്ച റോക്കറ്റിന്റെ വിക്ഷേപണം 18ന്

കോൺഗ്രസ് വിമർശന പോസ്റ്റ് പങ്കിട്ട് തരൂർ

Malayala Manorama

Rocket built by students of engineering college in Idukki all set for launch

It Was Just a rare mora

The Hindu

മലയാള മനോരമ

കുതിച്ചുയർന്ന് സ്വപ്നം...

Malayala Manorama

കേരള കൗമുദി

ക്യാമ്പസിൽ കരഘോഷമുയർന്നു, ബേസിലിയൻ 01 റോക്കറ്റ് കുതിച്ചുയർന്നു

Kerala Kaumudi

DEEPIKA

പീരുമേടിന്റെ സ്വന്തം റോക്കറ്റ് വിക്ഷേപണത്തിനൊരുങ്ങുന്നു

Deepika

FIG 15.2: Articles

CHAPTER 16

LAUNCH DAY GALLERY

FIG 16.1:Photos

CHAPTER 17
CONCLUSION

The Basilian-01 rocket avionics system is intended to be designed with a strong emphasis on reliability, redundancy, and autonomous operation. The project demonstrates an effective hardware architecture capable of monitoring flight parameters, maintaining long-range communication, performing onboard data logging, and executing safe recovery mechanisms without external intervention. By dividing the system into ground control and onboard avionics sections, the design ensures clear functional separation while maintaining continuous real-time telemetry during flight.

The integration of dual ESP32 flight controllers, redundant power regulation using multiple UBECs, and backup deployment mechanisms significantly enhances system robustness and fault tolerance. Critical sensors such as the MPU6050, BME280, and GPS module enable accurate acquisition of motion, environmental, and positional data, which are essential for flight behavior analysis and recovery logic. The use of LoRa-based RF communication at 433 MHz provides reliable long-range telemetry with low power consumption, making it well suited for sounding rocket applications.

Local data logging using an SD card ensures complete preservation of flight data even in the event of communication loss, enabling thorough post-flight analysis. The inclusion of primary and backup servo-based deployment systems further improves mission safety by minimizing the risk of recovery failure. Overall, the system is designed to operate autonomously while providing sufficient ground-level monitoring for mission assurance.

The Basilian-01 project is proposed as a cost-effective, modular, and redundant avionics system can be developed using commercially available components. The hardware architecture developed in this project can serve as a reliable platform for future enhancements such as advanced flight algorithms, additional atmospheric sensors, and experimental payload integration. Hence, the proposed avionics system is suitable for experimental rocket missions and provides a strong foundation for further research and development in amateur and academic rocketry.

REFERENCES

[1] **S. R. Nair, A. Thomas, and J. Mathew**, "*Development of a Servo-Based Model Rocket and Flight Behavior Analysis Using IMU Data*," in Proceedings of the IEEE International Conference on Aerospace Electronics and Remote Sensing Technology (ICARES), 2022.

[2] **M. D. Richard, L. A. Johnson, and P. K. Menon**, "*Design and Implementation of a Flight Computer for Sounding Rockets (S3FC)*," in IEEE Aerospace Conference, Big Sky, MT, USA, 2018.

[3] **F. Romano, G. Esposito, and L. Scarpato**, "*A Sounding Rocket as a Test Bench for Cost-Effective Aerospace Measurements*," in IEEE Transactions on Aerospace and Electronic Systems, vol. 55, no. 4, pp. 1821–1832, Aug. 2019.

[4] **J. C. Smith and R. K. Brown**, "*Low-Cost Avionics Systems for Small-Scale Rocket Applications*," in IEEE Aerospace Conference, Big Sky, MT, USA, 2017.

[5] **A. Kumar and S. Verma**, "*Design of Redundant Avionics Systems for Experimental Rockets*," in IEEE International Conference on Advanced Communication Systems and Computing, 2020.

[6] **T. Noguchi and A. G. Ramonet**, "*Node Replacement Method for Disaster-Resilient Wireless Sensor Networks*," in 10th Annual Computing and Communication Workshop and Conference (CCWC), Las Vegas, NV, USA, 2020, pp. 0789–0795.

[7] **H. Zhang, Y. Liu, and M. Chen**, "*Implementation of LoRa-Based Telemetry Systems for Long-Range Aerospace Applications*," in IEEE Access, vol. 8, pp. 145321–145330, 2020.

[8] **R. Patel and D. Shah**, "*Embedded Sensor Fusion Techniques for Flight State Estimation in Small Rockets*," in IEEE Sensors Journal, vol. 21, no. 10, pp. 11245–11254, May 2021.

[9] **M. L. Pereira, J. A. Sousa, and P. R. Silva**, "*Design and Validation of Redundant Recovery Systems for Model Rockets*," in IEEE International Symposium on Safety, Security, and Rescue Robotics, 2019.

[10] **K. Anderson and L. Thompson**, "*Power Management and Reliability Enhancement in Embedded Aerospace Systems*," in IEEE Transactions on Aerospace and Electronic Systems, vol. 56, no. 2, pp. 1345–1356, Apr. 2020.

APPENDICES

A. Hardware Component Pinout

A.1 ESP32-A I/O Configuration

Function	GPIO Pin	Bus/Protocol
BME280-A SDA	GPIO 21	I^2C
BME280-A SCL	GPIO 22	I^2C
MPU6050 SDA	GPIO 21	I^2C (shared)
MPU6050 SCL	GPIO 22	I^2C (shared)
GPS RX (from NEO-6M TX)	GPIO 16	UART1
GPS TX (to NEO-6M RX)	GPIO 17	UART1
LoRa SX1278 CS	GPIO 5	SPI
LoRa SX1278 MOSI	GPIO 23	SPI
LoRa SX1278 MISO	GPIO 19	SPI
LoRa SX1278 CLK	GPIO 18	SPI
LoRa SX1278 RST	GPIO 14	GPIO
LoRa SX1278 DIO0	GPIO 26	GPIO (interrupt)
Servo-A PWM	GPIO 12	GPIO
SD Card CS	GPIO 4	SPI (shared)
Battery ADC	GPIO 35	ADC
Unused	GPIO 2, 27, 25, 32, 33	GPIO

A.2 ESP32-B I/O Configuration

Function	GPIO Pin	Bus/Protocol
BME280-B SDA	GPIO 21	I^2C
BME280-B SCL	GPIO 22	I^2C
MPU6050 SDA	GPIO 21	I^2C (shared)
MPU6050 SCL	GPIO 22	I^2C (shared)
GPS RX (from NEO-6M TX)	GPIO 16	UART1
GPS TX (to NEO-6M RX)	GPIO 17	UART1
LoRa SX1278 CS	GPIO 5	SPI
LoRa SX1278 MOSI	GPIO 23	SPI
LoRa SX1278 MISO	GPIO 19	SPI
LoRa SX1278 CLK	GPIO 18	SPI
LoRa SX1278 RST	GPIO 14	GPIO
LoRa SX1278 DIO0	GPIO 26	GPIO (interrupt)
Servo-B PWM	GPIO 13	GPIO

Function	GPIO Pin	Bus/Protocol
SD Card CS	GPIO 4	SPI (shared)
Battery ADC	GPIO 35	ADC
Unused	GPIO 2, 27, 25, 32, 33	GPIO

B. Telemetry Protocol Definition

B.1 CSV Field Order (

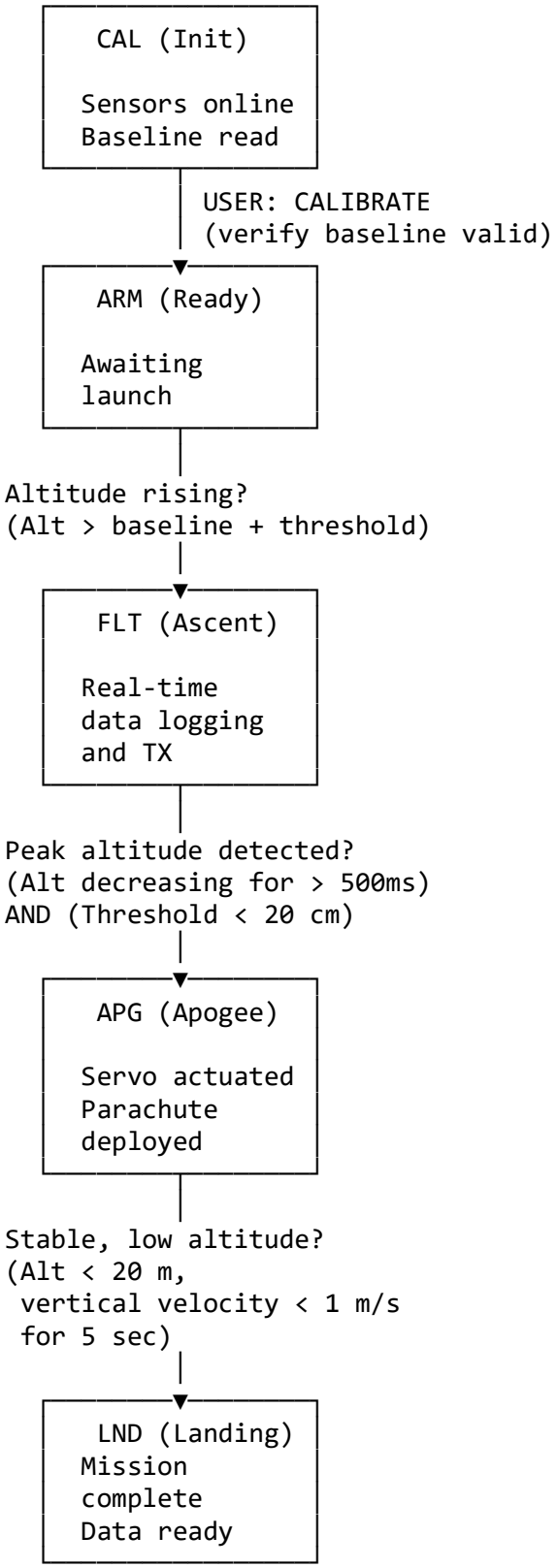
All telemetry packets follow this exact order:

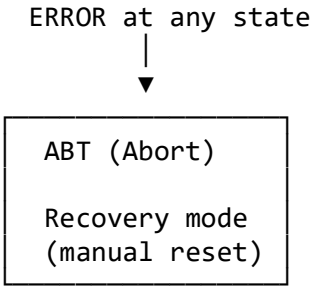
```
1. STATE (string)
2. temp (float, °C)
3. hum (float, %)
4. press (float, hPa)
5. altCm (float, cm) [Effective altitude]
6. battPct (float, %)
7. pitch (float, degrees)
8. roll (float, degrees)
9. yaw (float, degrees)
10. ax (float, m/s²)
11. ay (float, m/s²)
12. az (float, m/s²)
13. gx (float, rad/s)
14. gy (float, rad/s)
15. gz (float, rad/s)
16. lat (float, degrees)
17. lon (float, degrees)
18. gpsFix (int: 0/1/2)
19. sats (int, count)
20. hdop (float, m)
21. speed_kmph (float, km/h)
22. timestamp (long, Unix epoch IST)
[RSSI added by ground station: int, dBm]
```

B.2 LoRa Link Configuration

Parameter	Value
Frequency	433Mhz
Modulation	LoRa (chirp spread spectrum)
Bandwidth	125 kHz
Spreading Factor	SF9
Coding Rate	4/5
Preamble Length	8 symbols
Max Packet Size	255 bytes
TX Power	+17 dBm

C. Flight State Diagram (Detailed)





D. Main Flight Loop (Both ESP32s)

```
// Dual-ESP32 Flight Control Loop
// Both ESP32-A and ESP32-B execute identical firmware

void setup() {
  Serial.begin(115200);
  initSensors();        // BME280, MPU6050, GPS, LoRa
  initSD();             // SD card for logging
  initServo();          // Parachute servo PWM
  initWatchdog();       // 5-second watchdog

  flightState = CAL;    // Start in calibration state
}

void loop() {
  esp_task_wdt_reset(); // Feed watchdog

  // 1. SENSOR ACQUISITION
  readBME280();         // Altitude, temp, humidity
  readMPU6050();        // Acceleration, rotation
  readGPS();            // Position, timestamp
  readBattery();        // Voltage → battery %

  // 2. ALTITUDE VOTING
  // Local copy of altitude from this controller
  localAltitude = calculateAltitude(bme280_pressure);
  // Note: Inter-ESP32 voting happens via shared variables
  //       or LoRa link (implementation-specific)

  // 3. FLIGHT STATE MACHINE
  switch (flightState) {
    case CAL:
      handleCalibration();   // Read baseline, validate sensors
      break;

    case ARM:
      handleArmed();         // Wait for launch
      break;
```

```
    case FLT:
      handleFlight();         // Ascent logic
      // Check for apogee
      if (apogeeDetected()) {
        flightState = APG;
        deployParachute();    // Servo command
      }
      break;

    case APG:
      handleApogee();         // Parachute descent
      // Check for landing
      if (landingDetected()) {
        flightState = LND;
      }
      break;

    case LND:
      handleLanding();        // Mission complete, safe to power down
      break;

    case ABT:
      handleAbort();          // Error recovery
      break;
  }

  // 4. TELEMETRY GENERATION
  generateCSVTelemetry();

  // 5. LoRa TX
  if (loraTimer > TELEMETRY_INTERVAL) {
    transmitVialoRa(telemetryCSV);
    loraTimer = 0;
  }

  // 6. SD CARD LOGGING
  logToSD(telemetryCSV);    // Non-blocking

  // 7. GROUND COMMAND PROCESSING
  if (loraReceived()) {
    String cmd = loraRead();
    processCommand(cmd);      // E.g., RELEASE, HOLD, etc.
  }

  delay(10);  // Main loop iteration time
}

bool apogeeDetected() {
```

```
  // Voting logic: Use effective altitude = max(Alt_A, Alt_B)
  // This function assumes Inter-ESP32 altitude sharing is implemented
  float effectiveAltitude = getEffectiveAltitude();

  if (flightState != FLT) return false;
  if (effectiveAltitude < previousAltitude - APOGEE_THRESHOLD) {
    if (flightDuration > MIN_FLIGHT_TIME) {
      return true;
    }
  }
  previousAltitude = effectiveAltitude;
  return false;
}

void deployParachute() {
  // Servo command: 2000 µs pulse (180°)
  servoPWM.write(180);
  delay(500);  // Wait for servo to move

  // Log deployment
  Serial.println("PARACHUTE DEPLOYED");
}

void processCommand(String cmd) {
  if (cmd == "PING") {
    // No response required
  }
  else if (cmd.startsWith("PING,")) {
    String token = cmd.substring(5);
    transmitVialoRa("ACK_PING," + token);
  }
  else if (cmd == "RELEASE") {
    // Test parachute deployment (only if safe state)
    if (flightState == ARM || flightState == LND) {
      deployParachute();
      transmitVialoRa("ACK_RELEASE");
    }
  }
  else if (cmd == "HOLD") {
    // Lock servo in place
    servoPWM.write(currentServoAngle);
    transmitVialoRa("ACK_HOLD");
  }
  else if (cmd.contains(",")) {
    // Manual servo control: "angle1,angle2"
    int angle1 = cmd.substring(0, cmd.indexOf(",")).toInt();
    servoPWM.write(angle1);
    transmitVialoRa("ACK_ANGLE");
```

```
  }
}
```

E. Ground Station

```
// Ground Station Display Software
// Single ESP32 with LoRa RX and command TX interface

void setup() {
  Serial.begin(115200);        // USB to PC
  initLoRa();                  // LoRa RX/TX
  initBuzzer();                // Audible feedback

  Serial.println("Ground Station Ready");
}

void loop() {
  // 1. TELEMETRY RECEPTION
  if (LoRa.parsePacket()) {
    String telemetry = LoRa.readString();
    int rssi = LoRa.packetRssi();

    // Append RSSI
    telemetry += "," + String(rssi);

    // Display
    Serial.println(telemetry);

    // Audible feedback
    buzzer.beep(100);
  }

  // 2. COMMAND TRANSMISSION (from Serial input)
  if (Serial.available()) {
    String userInput = Serial.readStringUntil('\n');

    if (validateCommand(userInput)) {
      transmitCommand(userInput);
      buzzer.beep(150);  // TX alert

      // Wait for ACK (timeout 1 second)
      unsigned long ackTimeout = millis() + 1000;
      while (millis() < ackTimeout) {
        if (LoRa.parsePacket()) {
          String ack = LoRa.readString();
          if (ack.startsWith("ACK_")) {
            Serial.println("ACK: " + ack);
            buzzer.beep(50);
            break;
          }
```

```
        }
      }
    }
    else {
      Serial.println("ERROR: Invalid command");
    }
  }

  delay(10);
}

void transmitCommand(String cmd) {
  LoRa.beginPacket();
  LoRa.print(cmd);
  LoRa.endPacket();

  Serial.println("SENT: " + cmd);
}

bool validateCommand(String cmd) {
  // Whitelist of valid commands
  if (cmd == "PING") return true;
  if (cmd.startsWith("PING,")) return true;
  if (cmd == "RELEASE") return true;
  if (cmd == "HOLD") return true;
  if (cmd.matches("^\\d{1,3},\\d{1,3}$")) return true; // angle1,angle2

  return false;
}
```

ANNEXURE – COMPLETE SYSTEM CODE

A. ESP32 ONBOARD FLIGHT CONTROLLER

```
ON BOARD CODE

// Combined final ON-BOARD ESP32 firmware
// LoRa + BME280 + MPU6050 + GPS (Serial2) + SD logging + Servos + Battery + PING/ACK
// Timestamp field appended as IST (India Standard Time, UTC+5:30)
// *** ADDED: ESP-NOW for redundant data link to ESP32-B Logger ***

#include <SPI.h>
#include <LoRa.h>
#include <Wire.h>
#include <Adafruit_BME280.h>
#include <ESP32Servo.h>
#include <TinyGPSPlus.h>
#include <SD.h>
#include <WiFi.h>      // Required for ESP-NOW
#include <esp_now.h>   // Required for ESP-NOW

// ---------- Hardware pin config ----------
static const uint8_t LORA_SS    = 5;     // LoRa CS
static const uint8_t LORA_RST   = 14;    // LoRa RST
static const uint8_t LORA_DIO0  = 2;     // LoRa DIO0
static const long    LORA_FREQ  = 433E6;

static const int SPI_SCK    = 18;
static const int SPI_MISO   = 19;
static const int SPI_MOSI   = 23;

static const uint8_t SD_CS = 13;        // SD card CS

static const uint8_t BUZZER_PIN = 27;
static const uint8_t SERVO1_PIN = 25;
static const uint8_t SERVO2_PIN = 26;

static const uint8_t I2C_SDA = 21;
static const uint8_t I2C_SCL = 22;
static const uint8_t BME_ADDR = 0x77;
static const uint8_t MPU6050_ADDR = 0x68;

static const uint8_t BATT_ADC_PIN = 34;

// GPS (Serial2)
static const int GPS_RX_PIN = 16; // connect GPS TX -> GPIO16
static const int GPS_TX_PIN = 17; // optional: connect GPS RX -> GPIO17
static const unsigned long GPS_BAUD = 9600;

// ---------- Globals / objects ----------
Adafruit_BME280 bme;
Servo servo1, servo2;
TinyGPSPlus gps;

bool bmeOK = false;
bool imuOK = false;
```

```
53 bool sdOK = false;
54
55 File sdFile;
56
57 // ----- IMU calibration & orientation -----
58 float ax_off = 0, ay_off = 0, az_off = 0;
59 float gx_off = 0, gy_off = 0, gz_off = 0;
60 double ax_sum = 0, ay_sum = 0, az_sum = 0;
61 double gx_sum = 0, gy_sum = 0, gz_sum = 0;
62 long imu_samples = 0;
63 bool imuCalibrated = false;
64 float pitch = 0, roll = 0, yaw = 0;
65 unsigned long lastOrientationMs = 0;
66
67 // ----- Battery divider/calibration -----
68 static const float ADC_REF_V = 3.3f;
69 static const int ADC_MAX = 4095;
70 static const float R1 = 220000.0f;
71 static const float R2 = 100000.0f;
72 static const float CAL = 1.055f;
73 static const float BATT_FULL_V = 8.51f;
74 static const float BATT_EMPTY_V = 6.60f;
75
76 // ----- Flight timing, calibration, state machine -----
77 unsigned long lastTelemetry = 0;
78 static const unsigned long telemetryInterval = 1000;
79
80 static const unsigned long calibDuration = 20000; // 20s
81 unsigned long calibStartTime = 0;
82 bool altitudeCalibrated = false;
83 double altSum = 0.0;
84 long altSamples = 0;
85 float altBaselineMeters = 0.0f;
86
87 enum RocketState { STATE_CAL, STATE_ARMED, STATE_FLIGHT, STATE_APOGEE,
       STATE_LAND, STATE_ABORT };
88 RocketState currentState = STATE_CAL;
89 long lastAltCm = 0;
90 bool haveLastAlt = false;
91 long maxAltCm = 0;
92
93 static const long LIFTOFF_THRESH_CM     = 500;
94 static const long APOGEE_DROP_CM        = 50;
95 static const long LAND_TOLERANCE_CM     = 500;
96 static const long ABORT_ALT_THRESH_CM = 12000;
97
98 // ----- ACK behavior -----
99 static const int ACK_REPEATS = 2;
100 static const int ACK_DELAY_MS = 180;
101 static bool appendAckToNextTelemetry = false;
102 static String pendingAckToken = "";
103
104
105 // ******************************************************
106 // ---------- ESP-NOW specific configuration ----------
107
108 // ---------- ESP32-B MAC (SD LOGGER) ----------
109 uint8_t ESP32_B_MAC[] = {0xC0, 0xCD, 0xD6, 0x85, 0x6E, 0xCC};  //C0:CD:D6
```

```
    :85:6E:CC
110
111 // Packet structure for ESP-NOW (must be compatible with receiver)
112 typedef struct {
113     char csv[420];    // full telemetry CSV
114 } ESPNOW_Packet;
115
116 ESPNOW_Packet espNowPacket;
117
118
119 void initESPNOW() {
120     // ESP-NOW requires WiFi in STA mode to operate.
121     WiFi.mode(WIFI_STA);
122
123     if (esp_now_init() != ESP_OK) {
124         Serial.println("[ESP-NOW] Init FAILED");
125         return;
126     }
127
128     esp_now_peer_info_t peer{};
129     memcpy(peer.peer_addr, ESP32_B_MAC, 6);
130     peer.channel = 0; // Default channel
131     peer.encrypt = false; // No encryption
132
133     // Add peer to the list
134     if (esp_now_add_peer(&peer) != ESP_OK) {
135         Serial.println("[ESP-NOW] Peer add FAILED");
136         return;
137     }
138
139     Serial.println("[ESP-NOW] Ready -> ESP32-B");
140 }
141
142 // ******************************************************
143
144
145 // ---------- Helpers: buzzer, servos ----------
146 void beepTX(){ digitalWrite(BUZZER_PIN, HIGH); delay(50); digitalWrite(
    BUZZER_PIN, LOW); }
147 void beepRX(){ for(int i=0;i<2;i++){ digitalWrite(BUZZER_PIN,HIGH); delay
    (40); digitalWrite(BUZZER_PIN,LOW); delay(40);} }
148
149 void setServos(int a1,int a2){ a1 = constrain(a1,0,180); a2 = constrain(a2
    ,0,180); servo1.write(a1); servo2.write(a2); }
150 bool applyServoAngles(const String &cmd){ int c=cmd.indexOf(','); if(c<0)
    return false; String s1=cmd.substring(0,c), s2=cmd.substring(c+1); s1.
    trim(); s2.trim(); if(s1.isEmpty()||s2.isEmpty()) return false;
    setServos(s1.toInt(), s2.toInt()); return true; }
151 bool handleServoMessage(const String &msgRaw){ String m=msgRaw; m.trim();
    String up=m; up.toUpperCase(); if(up=="RELEASE"){ setServos(150,150);
    return true; } if(up=="HOLD"){ setServos(75,75); return true; } return
    applyServoAngles(m); }
152
153 // ---------- Battery ----------
154 float readBatteryVoltage(){ int raw = analogRead(BATT_ADC_PIN); float vAdc
    = raw * (ADC_REF_V / (float)ADC_MAX); float vBatt = vAdc * ((R1 + R2) /
    R2); vBatt *= CAL; return vBatt; }
155 int readBatteryPercent(){ float v = readBatteryVoltage(); v = constrain(v,
```

```
    BATT_EMPTY_V, BATT_FULL_V); float pct = (v - BATT_EMPTY_V) * 100.0f / (
    BATT_FULL_V - BATT_EMPTY_V); return (int)(constrain(pct,0.0f,100.0f)+0.5
    f); }
156
157 // ---------- Altitude calibration ----------
158 void updateAltitudeCalibration(float altMeters){
159     if(altitudeCalibrated || !bmeOK) return;
160     unsigned long now = millis();
161     altSum += altMeters; altSamples++;
162     if(now - calibStartTime >= calibDuration){
163         if(altSamples>0) altBaselineMeters = (float)(altSum / (double)
              altSamples);
164         else altBaselineMeters = altMeters;
165         altitudeCalibrated = true;
166         Serial.print("[CALIB] baseline m: "); Serial.println(
              altBaselineMeters,3);
167     }
168 }
169
170 // ---------- IMU low-level ----------
171 void imuWriteByte(uint8_t reg,uint8_t data){ Wire.beginTransmission(
    MPU6050_ADDR); Wire.write(reg); Wire.write(data); Wire.endTransmission()
    ; }
172 void imuReadBytes(uint8_t reg,uint8_t *buf,uint8_t len){ Wire.
    beginTransmission(MPU6050_ADDR); Wire.write(reg); Wire.endTransmission(
    false); Wire.requestFrom(MPU6050_ADDR,len); uint8_t i=0; while(Wire.
    available() && i<len) buf[i++]=Wire.read(); }
173
174 bool initIMU(){
175     imuWriteByte(0x6B,0x00); // wake
176     delay(50);
177     imuWriteByte(0x1C,0x00); // accel ±2g
178     imuWriteByte(0x1B,0x00); // gyro ±250 dps
179     return true;
180 }
181
182 void readIMU(float &ax,float &ay,float &az,float &gx,float &gy,float &gz){
183     uint8_t b[14]; imuReadBytes(0x3B,b,14);
184     int16_t rawAx = (b[0]<<8)|b[1]; int16_t rawAy=(b[2]<<8)|b[3]; int16_t
          rawAz=(b[4]<<8)|b[5];
185     int16_t rawGx = (b[8]<<8)|b[9]; int16_t rawGy=(b[10]<<8)|b[11]; int16_t
           rawGz=(b[12]<<8)|b[13];
186     ax = rawAx / 16384.0f; ay = rawAy / 16384.0f; az = rawAz / 16384.0f;
187     gx = rawGx / 131.0f; gy = rawGy / 131.0f; gz = rawGz / 131.0f;
188 }
189
190 // ---------- IMU calibration & orientation ----------
191 void updateIMUCalibration(float ax,float ay,float az,float gx,float gy,
    float gz){
192     if(imuCalibrated) return;
193     unsigned long now = millis();
194     ax_sum += ax; ay_sum += ay; az_sum += (az - 1.0f);
195     gx_sum += gx; gy_sum += gy; gz_sum += gz;
196     imu_samples++;
197     if(now - calibStartTime >= calibDuration){
198         if(imu_samples>0){
199             ax_off = ax_sum / imu_samples; ay_off = ay_sum / imu_samples;
                az_off = az_sum / imu_samples;
```

```
200             gx_off = gx_sum / imu_samples; gy_off = gy_sum / imu_samples;
                    gz_off = gz_sum / imu_samples;
201         }
202         imuCalibrated = true;
203         lastOrientationMs = now;
204         Serial.println("[IMU] calibrated");
205     }
206 }
207
208 void updateOrientation(float ax,float ay,float az,float gx,float gy,float
        gz,float dt){
209     const float alpha = 0.98f;
210     float pitchAcc = atan2f(ay, az) * 57.2957795f;
211     float rollAcc  = atan2f(-ax, sqrtf(ay*ay + az*az)) * 57.2957795f;
212     pitch = alpha * (pitch + gy * dt) + (1 - alpha) * pitchAcc;
213     roll  = alpha * (roll  + gx * dt) + (1 - alpha) * rollAcc;
214     yaw   += gz * dt;
215 }
216
217 // ---------- Flight state ----------
218 void updateFlightState(long altCm){
219     if(!haveLastAlt){ lastAltCm=altCm; maxAltCm=altCm; haveLastAlt=true; }
220     if(altCm>maxAltCm) maxAltCm = altCm;
221     switch(currentState){
222         case STATE_CAL: if(altitudeCalibrated && imuCalibrated) {
                currentState=STATE_ARMED; Serial.println("[STATE] -> ARM"); }
                break;
223         case STATE_ARMED: if(altCm >= LIFTOFF_THRESH_CM){ currentState=
                STATE_FLIGHT; Serial.println("[STATE] -> FLT"); } break;
224         case STATE_FLIGHT:
225             if(altCm >= ABORT_ALT_THRESH_CM){ currentState=STATE_ABORT;
                    Serial.println("[STATE] -> ABT"); }
226             else if(altCm < maxAltCm - APOGEE_DROP_CM){ currentState=
                    STATE_APOGEE; Serial.println("[STATE] -> APG"); setServos
                    (150,150); }
227             break;
228         case STATE_APOGEE: if(abs(altCm) <= LAND_TOLERANCE_CM){
                currentState=STATE_LAND; Serial.println("[STATE] -> LND"); }
                break;
229         case STATE_ABORT: if(abs(altCm) <= LAND_TOLERANCE_CM){ currentState
                =STATE_LAND; Serial.println("[STATE] ABT->LND"); } break;
230         case STATE_LAND: break;
231     }
232     lastAltCm = altCm;
233 }
234
235 const char* stateTag(RocketState s){
236     switch(s){ case STATE_CAL: return "CAL"; case STATE_ARMED: return "ARM"
            ; case STATE_FLIGHT: return "FLT"; case STATE_APOGEE: return "APG";
            case STATE_LAND: return "LND"; case STATE_ABORT: return "ABT";
            default: return "UNK"; }
237 }
238
239 // ---------- Utility: produce IST timestamp from GPS (YYYY-MM-DD HH:MM:SS)
        ----------
240 String getISTTimestampFromGPS(){
241     if(!gps.time.isValid() || !gps.date.isValid()) return String("N/A");
242
```

```
243     // GPS time/date are UTC. Convert to IST (UTC+5:30)
244     int hh = gps.time.hour();
245     int mm = gps.time.minute();
246     int ss = gps.time.second();
247     int dd = gps.date.day();
248     int mo = gps.date.month();
249     int yy = gps.date.year();
250 
251     mm += 30;
252     if(mm >= 60){ mm -= 60; hh += 1; }
253     hh += 5;
254     if(hh >= 24){
255         hh -= 24;
256         dd += 1;
257         // NOTE: we do not implement full calendar rollover (month/year)
                here.
258     }
259 
260     char buf[32];
261     sprintf(buf, "%04d-%02d-%02d %02d:%02d:%02d", yy, mo, dd, hh, mm, ss);
262     return String(buf);
263 }
264 
265 // ---------- SD helper ----------
266 void sdAppendLine(const String &line){
267     if(!sdOK) return;
268     File f = SD.open("/flight_log.csv", FILE_APPEND);
269     if(f){
270         f.println(line);
271         f.close();
272     } else {
273         sdOK = false;
274         Serial.println("[SD] write failed, disabling SD writes");
275     }
276 }
277 
278 // ---------- Telemetry builder & sender (adds IST timestamp) ----------
279 void sendTelemetry(){
280     if(!bmeOK) return;
281 
282     float temp = bme.readTemperature();
283     float hum  = bme.readHumidity();
284     float pres = bme.readPressure() / 100.0f;
285     float altMeters = bme.readAltitude(1013.25f);
286 
287     if(!altitudeCalibrated) updateAltitudeCalibration(altMeters);
288 
289     long altCm = altitudeCalibrated ? (long)((altMeters - altBaselineMeters
        ) * 100.0f) : 0;
290     updateFlightState(altCm);
291     int battPct = readBatteryPercent();
292 
293     float axv=0,ayv=0,azv=0,gxv=0,gyv=0,gzv=0;
294     if(imuOK){
295         readIMU(axv,ayv,azv,gxv,gyv,gzv);
296         if(!imuCalibrated){
297             updateIMUCalibration(axv,ayv,azv,gxv,gyv,gzv);
298             axv=ayv=azv=gxv=gyv=gzv=0;
```

```
299         } else {
300             axv -= ax_off; ayv -= ay_off; azv -= az_off;
301             gxv -= gx_off; gyv -= gy_off; gzv -= gz_off;
302             unsigned long now = millis();
303             float dt = (lastOrientationMs==0) ? 0.01f : ((now -
                    lastOrientationMs) / 1000.0f);
304             if(dt <= 0) dt = 0.01f;
305             updateOrientation(axv,ayv,azv,gxv,gyv,gzv,dt);
306             lastOrientationMs = now;
307         }
308     }
309 
310     // GPS fields
311     bool gpsFix = gps.location.isValid();
312     double gpsLat = gpsFix ? gps.location.lat() : 0.0;
313     double gpsLon = gpsFix ? gps.location.lng() : 0.0;
314     int gpsSats = gps.satellites.isValid() ? gps.satellites.value() : 0;
315     float gpsHdop = gps.hdop.isValid() ? gps.hdop.hdop() : 0.0f;
316     float gpsSpeedKmph = gps.speed.isValid() ? gps.speed.kmph() : 0.0f;
317 
318     // IST timestamp (from GPS); fallback = "N/A"
319     String istTs = getISTTimestampFromGPS();
320 
321     // Build telemetry CSV payload (no spaces), fields (22 total):
322     // 0:STATE, 1:temp, 2:hum, 3:press, 4:altCm, 5:battPct,
323     // 6:pitch, 7:roll, 8:yaw,
324     // 9:ax, 10:ay, 11:az, 12:gx, 13:gy, 14:gz,
325     // 15:lat, 16:lon, 17:gpsFix, 18:sats, 19:hdop, 20:speed_kmph, 21:
            timestamp
326     String pkt; pkt.reserve(400);
327     pkt  = stateTag(currentState);
328     pkt += ',' + String(temp,2);
329     pkt += ',' + String(hum,2);
330     pkt += ',' + String(pres,2);
331     pkt += ',' + String(altCm);
332     pkt += ',' + String(battPct);
333     pkt += ',' + String(pitch,2);
334     pkt += ',' + String(roll,2);
335     pkt += ',' + String(yaw,2);
336     pkt += ',' + String(axv,3);
337     pkt += ',' + String(ayv,3);
338     pkt += ',' + String(azv,3);
339     pkt += ',' + String(gxv,3);
340     pkt += ',' + String(gyv,3);
341     pkt += ',' + String(gzv,3);
342     pkt += ',' + String(gpsLat,7);
343     pkt += ',' + String(gpsLon,7);
344     pkt += ',' + String(gpsFix ? 1 : 0);
345     pkt += ',' + String(gpsSats);
346     pkt += ',' + String(gpsHdop,2);
347     pkt += ',' + String(gpsSpeedKmph,2);
348 
349     // Append IST timestamp as last CSV field (22nd field)
350     pkt += ',' + istTs;
351 
352     // Append ACK if pending (this adds 2 extra fields, making 24 total)
353     if(appendAckToNextTelemetry && pendingAckToken.length() > 0){
354         pkt += ",ACK," + pendingAckToken;
```

```
355         appendAckToNextTelemetry = false;
356         pendingAckToken = "";
357     }
358 
359     // 1. LoRa send
360     LoRa.beginPacket();
361     LoRa.print(pkt);
362     LoRa.endPacket();
363     beepTX();
364 
365     // 2. ESP-NOW send (redundant link)
366     // Copy String to the fixed-size char array for ESP-NOW transmission
367     pkt.toCharArray(espNowPacket.csv, sizeof(espNowPacket.csv));
368     esp_now_send(ESP32_B_MAC, (uint8_t*)&espNowPacket, sizeof(espNowPacket)
        );
369 
370 
371     // 3. SD log: prepend timestamp as millis() and then the full pkt
372     String logline;
373     logline.reserve(pkt.length() + 20);
374     logline += String(millis());
375     logline += ',';
376     logline += pkt;
377     sdAppendLine(logline);
378 
379     // Serial debug (brief)
380     Serial.print("[TX␣TEL]␣");
381     Serial.println(pkt);
382 }
383 
384 // ---------- Setup ----------
385 void setup(){
386     Serial.begin(115200);
387     delay(200);
388     Serial.println("\n[ONBOARD]␣init");
389 
390     pinMode(BUZZER_PIN, OUTPUT); digitalWrite(BUZZER_PIN, LOW);
391     pinMode(BATT_ADC_PIN, INPUT);
392 
393     // I2C
394     Wire.begin(I2C_SDA, I2C_SCL);
395 
396     // BME
397     bmeOK = bme.begin(BME_ADDR);
398     if(bmeOK) Serial.println("BME280␣OK");
399     else Serial.println("BME280␣NOT␣FOUND");
400 
401     // IMU
402     imuOK = initIMU();
403     if(imuOK) Serial.println("MPU␣init␣(basic)");
404     else Serial.println("MPU␣init␣failed");
405 
406     // Servos
407     servo1.attach(SERVO1_PIN);
408     servo2.attach(SERVO2_PIN);
409     setServos(90,90);
410 
411     // SPI + LoRa
```

```
412     SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI, LORA_SS);
413     LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
414     if(!LoRa.begin(LORA_FREQ)){
415         Serial.println("LoRa␣init␣FAILED␣-␣check␣wiring␣and␣3.3V");
416         while(1){ delay(500); } // halt if LoRa essential
417     }
418     LoRa.setSpreadingFactor(7);
419     LoRa.setSignalBandwidth(125E3);
420     LoRa.setCodingRate4(5);
421     LoRa.setSyncWord(0x34);
422     Serial.println("LoRa␣OK");
423 
424     // ** Initialize ESP-NOW **
425     initESPNOW();
426 
427     // SD init (nonfatal)
428     if(SD.begin(SD_CS, SPI)){
429         sdOK = true;
430         Serial.println("SD␣initialized");
431         if(!SD.exists("/flight_log.csv")){
432             File h = SD.open("/flight_log.csv", FILE_WRITE);
433             if(h){
434                 // header columns
435                 h.println("ts_ms,STATE,temp,hum,press,altCm,battPct,pitch,
                        roll,yaw,ax,ay,az,gx,gy,gz,lat,lon,gpsFix,sats,hdop,
                        speed_kmph,IST");
436                 h.close();
437             }
438         }
439     } else {
440         sdOK = false;
441         Serial.println("SD␣init␣failed␣(continuing␣without␣SD)");
442     }
443 
444     // GPS Serial2
445     Serial2.begin(GPS_BAUD, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);
446     Serial.println("GPS␣serial2␣started");
447 
448     // calibration start
449     calibStartTime = millis();
450     altitudeCalibrated = false;
451     imuCalibrated = false;
452     appendAckToNextTelemetry = false;
453     pendingAckToken = "";
454 
455     Serial.println("Setup␣complete");
456 }
457 
458 // ---------- Main loop ----------
459 void loop(){
460     // read GPS bytes continuously
461     while(Serial2.available()) gps.encode(Serial2.read());
462 
463     unsigned long now = millis();
464     if(now - lastTelemetry >= telemetryInterval){
465         lastTelemetry = now;
466         sendTelemetry();
467     }
```

```
468
469     // LoRa command handling
470     int packetSize = LoRa.parsePacket();
471     if(packetSize){
472         String incoming;
473         while(LoRa.available()) incoming += (char)LoRa.read();
474         incoming.trim();
475         String upper = incoming; upper.toUpperCase();
476
477         // Handle PING without token
478         if(upper == "PING"){
479             LoRa.beginPacket(); LoRa.print("TXRX_OK"); LoRa.endPacket();
                    beepRX();
480             Serial.println("[RX␣CMD]␣PING␣->␣TXRX_OK");
481             return;
482         }
483
484         // Handle PING with token (sends ACK repeatedly)
485         if(upper.startsWith("PING,")){
486             int pos = incoming.indexOf(',');
487             if(pos >= 0){
488                 String token = incoming.substring(pos + 1); token.trim();
489                 if(token.length() > 0){
490                     for(int i=0;i<ACK_REPEATS;i++){
491                         delay(ACK_DELAY_MS);
492                         LoRa.beginPacket();
493                         LoRa.print("ACK," + token);
494                         LoRa.endPacket();
495                     }
496                     // Set flag to append ACK to the next telemetry packet
497                     appendAckToNextTelemetry = true;
498                     pendingAckToken = token;
499                     beepRX();
500                     Serial.print("[RX␣CMD]␣PING,␣token␣->␣ACK␣sent␣x");
                        Serial.println(ACK_REPEATS);
501                     return;
502                 }
503             }
504         }
505
506         // Handle servo/release commands
507         if(handleServoMessage(incoming)){
508             beepRX();
509             Serial.print("[RX␣CMD]␣servo␣->␣"); Serial.println(incoming);
510         } else {
511             Serial.print("[RX␣CMD]␣ignored␣->␣"); Serial.println(incoming);
512         }
513     }
514 }
```

Listing 1: Onboard ESP32 Flight Controller Firmware (LoRa + GPS + IMU + SD + ESP-NOW)

B. ESP32-B SD CARD LOGGER CODE

```

#include <WiFi.h>
#include <esp_now.h>
#include <SPI.h>
#include <SD.h>
#include <HardwareSerial.h>

// ================= SD PIN CONFIG =================
#define SD_CS   5
#define SD_SCK  18
#define SD_MISO 19
#define SD_MOSI 23

SPIClass sdSPI(VSPI);

// ================= SDS011 =================
#define SDS_RX 16
#define SDS_TX 17
HardwareSerial sdsSerial(2);

// ================= ESP32-A MAC =================
uint8_t ESP32_A_MAC[] = {0xC0,0xCD,0xD6,0x85,0x6E,0xCC};

// ================= STRUCT =================
typedef struct {
  char csv[420];
} ESPNOW_Packet;

volatile bool dataReady = false;
ESPNOW_Packet rxPacket;

// ================= SDS011 DATA =================
float pm25 = -1;
float pm10 = -1;

// ================= FILE =================
String logFile;

// ================= SDS011 READ =================
void readSDS011() {
  static uint8_t buf[10], idx = 0;

  while (sdsSerial.available()) {
    uint8_t b = sdsSerial.read();
    if (idx == 0 && b != 0xAA) continue;
    buf[idx++] = b;

    if (idx == 10) {
      idx = 0;
      if (buf[0] == 0xAA && buf[1] == 0xC0) {
        pm25 = (buf[3]*256 + buf[2]) / 10.0;
        pm10 = (buf[5]*256 + buf[4]) / 10.0;
      }
    }
  }
}
```

```
57
58 // ================= ESPNOW CALLBACK =================
59 void onReceive(const esp_now_recv_info *info,
60                const uint8_t *data,
61                int len) {
62
63   if (memcmp(info->src_addr, ESP32_A_MAC, 6) != 0) return;
64   if (len != sizeof(ESPNOW_Packet)) return;
65
66   memcpy(&rxPacket, data, sizeof(rxPacket));
67   dataReady = true;
68 }
69
70 // ================= SETUP =================
71 void setup() {
72   Serial.begin(115200);
73
74   WiFi.mode(WIFI_STA);
75
76   sdSPI.begin(SD_SCK, SD_MISO, SD_MOSI, SD_CS);
77   SD.begin(SD_CS, sdSPI);
78
79   logFile = "/log.csv";
80
81   File f = SD.open(logFile, FILE_WRITE);
82   f.println("DATA");
83   f.close();
84
85   sdsSerial.begin(9600, SERIAL_8N1, SDS_RX, SDS_TX);
86
87   esp_now_init();
88   esp_now_register_recv_cb(onReceive);
89 }
90
91 // ================= LOOP =================
92 void loop() {
93   readSDS011();
94
95   if (dataReady) {
96     dataReady = false;
97
98     String line = rxPacket.csv;
99     line += ",";
100    line += String(pm25);
101    line += ",";
102    line += String(pm10);
103
104    File f = SD.open(logFile, FILE_APPEND);
105    if (f) {
106      f.println(line);
107      f.close();
108    }
109  }
110 }
```

Listing 2: ESP32-B Logger (ESP-NOW Receiver + SDS011 + SD Storage)

C. GROUND STATION ESP32 CODE

```

#include <SPI.h>
#include <LoRa.h>

// ----- LoRa Pins -----
#define LORA_SS    5
#define LORA_RST   14
#define LORA_DIO0  2
#define LORA_FREQ  433E6

// ----- Buzzer -----
#define BUZZER_PIN 27

// ----- Expected telemetry field count from onboard -----
#define TELEMETRY_FIELDS 22   // without RSSI (it will be added as 23rd field)

// --------------------- BUZZER -------------------------
void beepTX() { digitalWrite(BUZZER_PIN, HIGH); delay(40); digitalWrite(BUZZER_PIN, LOW); }
void beepRX() { digitalWrite(BUZZER_PIN, HIGH); delay(40); digitalWrite(BUZZER_PIN, LOW); }

// -------------- Validate servo angle commands ----------
bool isValidServoAngles(const String &cmd) {
  int c = cmd.indexOf(',');
  if (c < 0) return false;
  String a = cmd.substring(0, c);
  String b = cmd.substring(c + 1);
  a.trim(); b.trim();
  // Check if both parts exist (length > 0)
  return (a.length() > 0 && b.length() > 0);
}

// ---------------------- CSV Splitter --------------------
// Splits the data string into the fields array, up to maxFields.
// If the data string has more than maxFields, the extras are ignored but not counted.
int splitCSV(const String &data, String *fields, int maxFields) {
  int count = 0;
  int start = 0;

  for (int i = 0; i <= data.length(); i++) {
    if (i == data.length() || data[i] == ',') {
      if (count < maxFields)
        fields[count++] = data.substring(start, i);
      start = i + 1;
    }
  }
  return count;
}

// ====================== SETUP ============================
void setup() {
  Serial.begin(115200);
```

```
  delay(700);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  // LoRa SPI pins (SCK=18, MISO=19, MOSI=23)
  SPI.begin(18, 19, 23, LORA_SS);
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);

  Serial.println("GROUND ESP32 STARTED");

  if (!LoRa.begin(LORA_FREQ)) {
    Serial.println("LORA INIT FAILED - CHECK MODULE");
    while (1);
  }
  Serial.println("LORA INIT OK");

  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setSyncWord(0x34);

  // NOTE: Header order corrected to match the On-Board telemetry output order
  Serial.println("STATE,temp,hum,press,altCm,battPct,pitch,roll,yaw,ax,ay,az,gx,gy,gz,lat,lon,gpsFix,sats,hdop,speed_kmph,timestamp,rssi");
}

// ====================== LOOP ============================
void loop() {

  // -------------------- SEND COMMANDS --------------------
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd.length() == 0) return;

    String up = cmd; up.toUpperCase();
    bool sendIt = false;

    // PING - send a simple PING or PING,token
    if (up.startsWith("PING")) {
      sendIt = true;
    }
    // Standard commands
    else if (up == "RELEASE" || up == "HOLD" || isValidServoAngles(cmd)) {
      sendIt = true;
    }

    if (sendIt) {
      LoRa.beginPacket();
      LoRa.print(cmd);
      LoRa.endPacket();
      Serial.print("SENT,"); Serial.println(cmd);
      beepTX();
    }
  }

```

```
109   // -------------------- RECEIVE PACKETS --------------------
110   int packetSize = LoRa.parsePacket();
111   if (!packetSize) return;
112
113   String incoming = "";
114   while (LoRa.available()) incoming += (char)LoRa.read();
115   incoming.trim();
116   int rssi = LoRa.packetRssi();
117
118   // ---------- Handle ACK responses (packets that are ONLY ACK) ----------
119   if (incoming == "TXRX_OK") {
120     Serial.print("TXRX_OK,"); Serial.println(rssi);
121     beepRX();
122     return;
123   }
124   if (incoming.startsWith("ACK,")) {
125     Serial.print(incoming); Serial.print(","); Serial.println(rssi);
126     beepRX();
127     return;
128   }
129
130   // ---------- Handle Telemetry ----------
131   // We use TELEMETRY_FIELDS + 1 for the array size to store the 22 fields
        + RSSI.
132   String fields[TELEMETRY_FIELDS + 1];
133
134   // splitCSV only fills up to TELEMETRY_FIELDS (22)
135   int count = splitCSV(incoming, fields, TELEMETRY_FIELDS);
136
137   // CRITICAL FIX: Ensure we got at least the expected number of fields.
138   // If the On-Board appends ACK data, count will still be 22, and we
        proceed.
139   if (count < TELEMETRY_FIELDS) {
140     // corrupted / incomplete packet, ignore
141     return;
142   }
143
144   // Attach RSSI as last field (at index 22, since fields[0] to fields[21]
        are the data)
145   fields[TELEMETRY_FIELDS] = String(rssi);
146
147   // Print full CSV line (23 fields: 22 data + 1 rssi)
148   for (int i = 0; i <= TELEMETRY_FIELDS; i++) {
149     Serial.print(fields[i]);
150     if (i < TELEMETRY_FIELDS) Serial.print(",");
151   }
152   Serial.println();
153
154   beepRX();
155 }
```

Listing 3: Ground Station (LoRa Receiver + Command TX + Telemetry Display)